REVIEW – III · SUBMISSION

The Working System and its Live Databases

GraphRAG-Powered Cyber Threat Intelligence Platform: A Multi-Model NoSQL Approach

Deliverable	Review – III (10 marks): final demonstration and innovation
Team	Shivansh Sharma (23BCE0254) Himali Sharma (23BCE0317) Aayush Sinha (23BCE0558)
Institution	VIT Vellore
Guide	Dr. Lydia Jane G
Repository	github.com/Shivansharmaaaa/graphrag
Evidence captured	30 October 2026, 20:57–21:15 IST, on the running system

About this document

Every screenshot here was taken on 30 October 2026 from the running platform and its four NoSQL databases (Neo4j, MongoDB, Redis and Qdrant). The threat intelligence in them was downloaded the same day from MITRE ATT&CK, abuse.ch Feodo Tracker and Emerging Threats. Screenshots are unedited: they are only scaled and framed. Terminal output is copied verbatim. All 33 original screenshots and every log are in `review3/submission/`.

The design, the full performance evaluation and the operating guide are in the Final Report, the User Manual and the slides submitted alongside this document (Section 8).

Contents

Section	Page
1. Project Summary	3
2. System Under Demonstration	5
3. Working System: Analyst Console	7
4. Working System: REST and Enforcement API	19
5. Live Databases	21
6. One Indicator Across the Four Databases	30
7. Tests and Evaluation on the Live System	31
8. Submission Contents	33
Appendix A. Reproducing This Document	33

Figures are screenshots of the live system unless a caption says otherwise (Figure 1 is a diagram). Tables are numbered in order of appearance.

1. Project Summary

Cyber threat intelligence (CTI) is polyglot data. Relationships between threat actors, campaigns, malware and infrastructure form a graph. Analyst reports are documents whose fields vary from report to report. Blocked indicators of compromise (IOCs) need millisecond lookups and must expire on time. Analysts ask questions in natural language, which needs semantic search. One database serves all of these poorly.

The platform gives each workload the NoSQL engine built for it, and adds a GraphRAG layer on top. A question is answered by retrieving evidence from all four stores: similar report passages from Qdrant, multi-hop paths from Neo4j, the full reports from MongoDB and the live blocklist state from Redis. A locally hosted LLM (llama3.2:3b through Ollama) then writes the answer. A groundedness gate removes every sentence that does not cite a graph edge, a passage, a report or an IOC state. Traffic Light Protocol (TLP) clearance is enforced on every read path. Firewalls and EDR tools query a fail-closed enforcement API, and API requests that arrive from a blocklisted address are refused.

Store	Engine (version)	What it holds	Contents on 30 October 2026
Graph	Neo4j 5.26.31 Community	Threat knowledge graph: actors, campaigns, malware, techniques, infrastructure, indicators, reports	2,223 nodes, 18,491 relationships
Document	MongoDB 7.0.43	Reports, STIX bundles, users, audit log, ingest log, blocked-request log	250 reports in 7 collections
Key–value	Redis 7.4.11	IOC blocklist `ioc:{type}:{value}`, one TTL per key	565 live IOC keys
Vector	Qdrant 1.13.4	Report-chunk embeddings (64-dim, cosine) with payload indexes	796 chunks
LLM	Ollama 0.5.4, llama3.2:3b	Writes the answer from retrieved evidence; runs locally	—

Table 1. The four databases and the model, with the data they held when the screenshots were taken. Counts are from the databases' own command-line tools (Section 5).

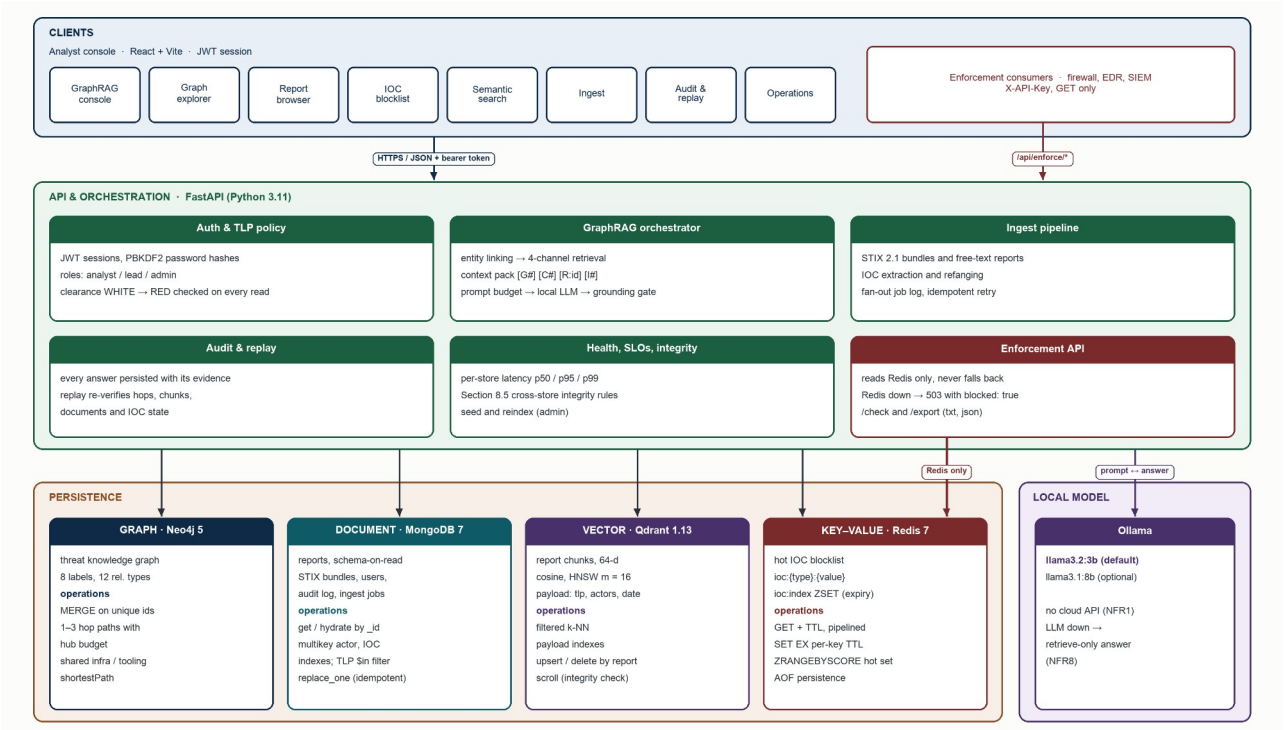


Figure 1. Implemented architecture (diagram): analyst console and enforcement clients, the FastAPI service, the four NoSQL stores and the local LLM.

Data. The graph holds 178 threat actors, 843 malware and tools, 227 techniques and 63 campaigns, mostly from MITRE ATT&CK Enterprise v20.0. The blocklist holds the 21 botnet command-and-control (C2) servers on abuse.ch Feodo Tracker (QakBot 16, Emotet 5) and 536 compromised hosts listed by Emerging Threats. All three feeds were downloaded on 30 October 2026. Alongside them sit the project's 12 seed analyst reports and one STIX 2.1 bundle that was ingested during the demonstration (Figure 15).

2. System Under Demonstration

2.1 Environment

Component	Version used for this document
Host	macOS (Darwin 25.3), Docker 29.5.2; the four databases run as containers
API service	Python 3.11.7, FastAPI 0.116.1, Uvicorn 0.35.0 on port 8000
Database drivers	neo4j 5.28.1, pymongo 4.13.2, redis-py 6.2.0, qdrant-client 1.14.3
Analyst console	React 18.3.1 and Vite 5.4.11 on port 5173
Language model	Ollama 0.5.4 serving llama3.2:3b on the same machine (no cloud API)
Database consoles	Neo4j Browser (7474), mongo-express (8081), Redis Commander (8082, read-only), Qdrant dashboard (6333)
Screenshots	Google Chrome, headless, 1440 × 900 viewport at 2× scale, driven by `review3/capture_submission.js`

Table 2. Software and versions on the demonstration machine.

2.2 Starting the system

The system was started with the project's own make targets. The API was started with `TRUST_FORWARDED_FOR=true` so that the IP gate can be demonstrated from one machine by sending an `X-Forwarded-For` header (Section 4).

```
make infra              # Neo4j, MongoDB, Redis and Qdrant containers
make dbui               # mongo-express and Redis Commander
make import-feeds       # today's MITRE ATT&CK, Feodo Tracker and Emerging Threats data
cd backend && TRUST_FORWARDED_FOR=true ../.venv/bin/uvicorn app.main:app --port 8000
make web                # analyst console on http://localhost:5173
```

2.3 Running containers

```
$ docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}' | grep -v cloud   #
    2026-10-30 21:06 IST
NAMES                   IMAGE                                    STATUS
cti-mongo-express       mongo-express:1.0.2-20-alpine3.19        Up 3 minutes
cti-redis-commander     ghcr.io/joeferner/redis-commander:0.9.1  Up 4 days (healthy)
cti-redis               redis:7-alpine                           Up 4 days (healthy)
cti-qdrant              qdrant/qdrant:v1.13.4                    Up 4 days (healthy)
cti-mongo               mongo:7                                  Up 4 days (healthy)
cti-neo4j               neo4j:5.26-community                     Up 4 days (healthy)
```

2.4 Loading today's threat intelligence

The importer downloads the three public feeds and writes them into all four stores in one run. ATT&CK groups, software, techniques and campaigns go into Neo4j as nodes and relationships. MongoDB gets one report per group, campaign and feed. Every report is re-chunked and re-embedded into Qdrant. The feed IP addresses go into Redis, each with a TTL that depends on its severity.

```
cd backend && ../.venv/bin/python -m scripts.import_feeds
1/5 feeds
  downloading https://raw.githubusercontent.com/mitre-attack/attack-stix-data/master/enter
    prise-attack/enterprise-attack.json
  downloading https://feodotracker.abuse.ch/downloads/ipblocklist.json
  downloading https://feodotracker.abuse.ch/downloads/ipblocklist_aggressive.csv
  downloading https://rules.emergingthreats.net/blockrules/compromised-ips.txt
2/5 Neo4j      ATT&CK 20.0: mapping onto the existing graph
    57 Campaign, 178 ThreatActor, 842 Malware, 227 Technique; 16678 ATT&CK relationships;
    graph now 2216 nodes, 18478 relationships
3/5 MongoDB    237 reports upserted
4/5 Redis      557 feed IPs on the blocklist ([('Emerging Threats compromised hosts',
    536), ('abuse.ch Feodo Tracker', 21)])
5/5 Qdrant     refitting the embedder and re-indexing every report
    794 chunks
done in 28 s — restart the API so it loads the refitted embedder
```

2.5 Health of the running system

```
$ curl -s localhost:8000/api/health   # .stores, one line per store
  neo4j    ok=True  2216 nodes · localhost  (5.55 ms)
  mongodb  ok=True  249 reports · localhost  (5.74 ms)
  redis    ok=True  563 live IOCs · localhost  (1.18 ms)
  qdrant   ok=True  server: threat_chunks (794 chunks) · localhost  (11.38 ms)
  llm      ok=True  ollama · llama3.2:3b  (19.63 ms)
```

This check ran at 20:58, before the STIX ingest of Section 3.7 added 7 nodes, 1 report, 2 chunks and 2 IOCs. `/api/health` reaches every store and the model on each call. The same check drives the green status dots at the bottom left of every console page and the Operations page (Figure 20).

3. Working System: Analyst Console

The analyst console is a React single-page application that talks only to the API. Each user signs in with a role and a TLP clearance, and the API applies that clearance to every request. The screenshots below follow one analyst session (user `analyst`, clearance TLP:AMBER), then a junior analyst (TLP:GREEN) and a CTI lead (TLP:RED).

3.1 Sign-in

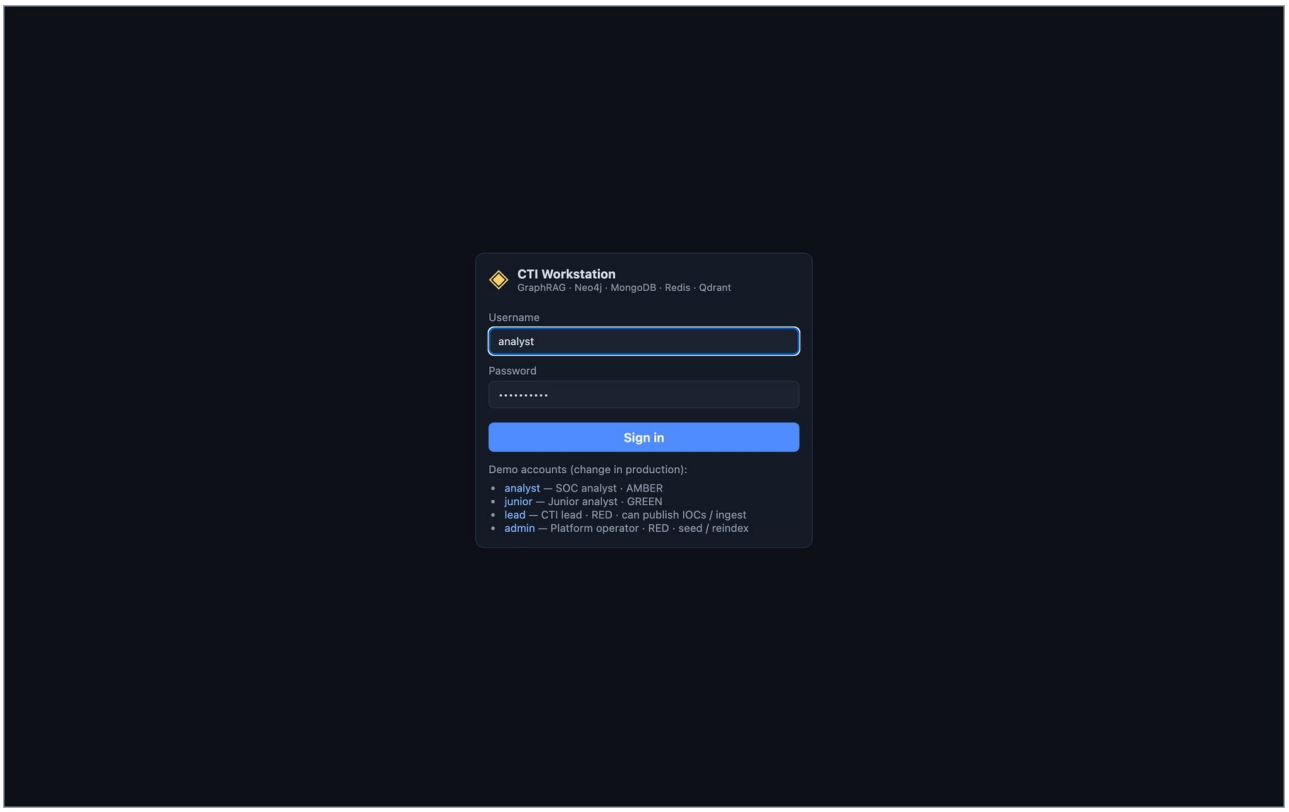


Figure 2. Sign-in page. Four demonstration accounts cover the roles: analyst (AMBER), junior (GREEN), lead (RED, can ingest and publish IOCs) and admin (operator).

3.2 GraphRAG question answering

The question in Figure 3 is about today's data: which botnet C2 servers host QakBot, and are they still blocked. Entity linking maps "QakBot" to the ATT&CK software node S0650. The graph expansion finds the Feodo Tracker servers that host it, and the IOC check reads each address's live state from Redis. Every sentence in the answer carries citation chips: **G** for a graph edge, **C** for a passage, **D** for a report, **I** for an IOC state. Three uncited sentences were removed by the groundedness gate. This is shown under the answer and can be expanded.

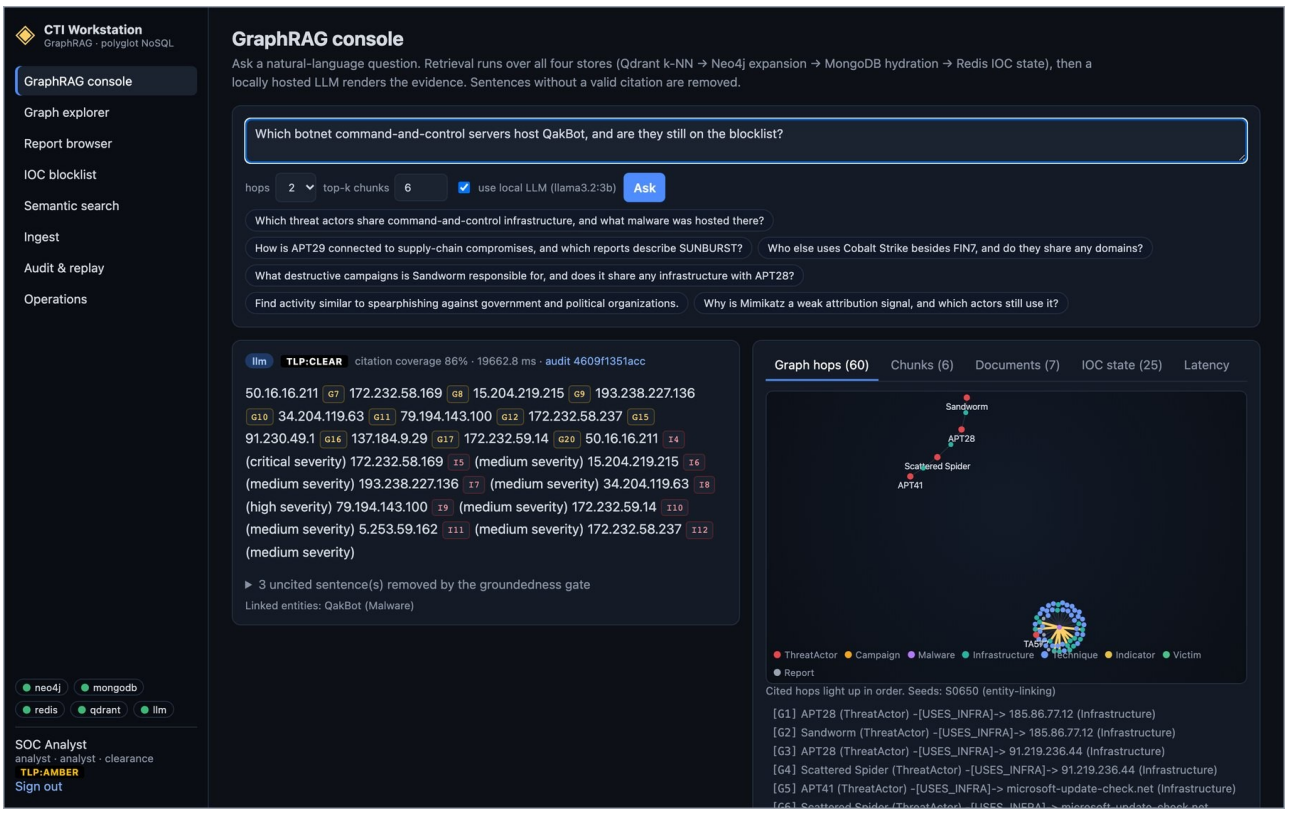


Figure 3. GraphRAG answer about QakBot C2 servers on today's data: llama3.2:3b, citation coverage 86%, 19.7 s end to end, with audit id. Each address is cited to its HOSTS edge (G) and its live blocklist entry (I).

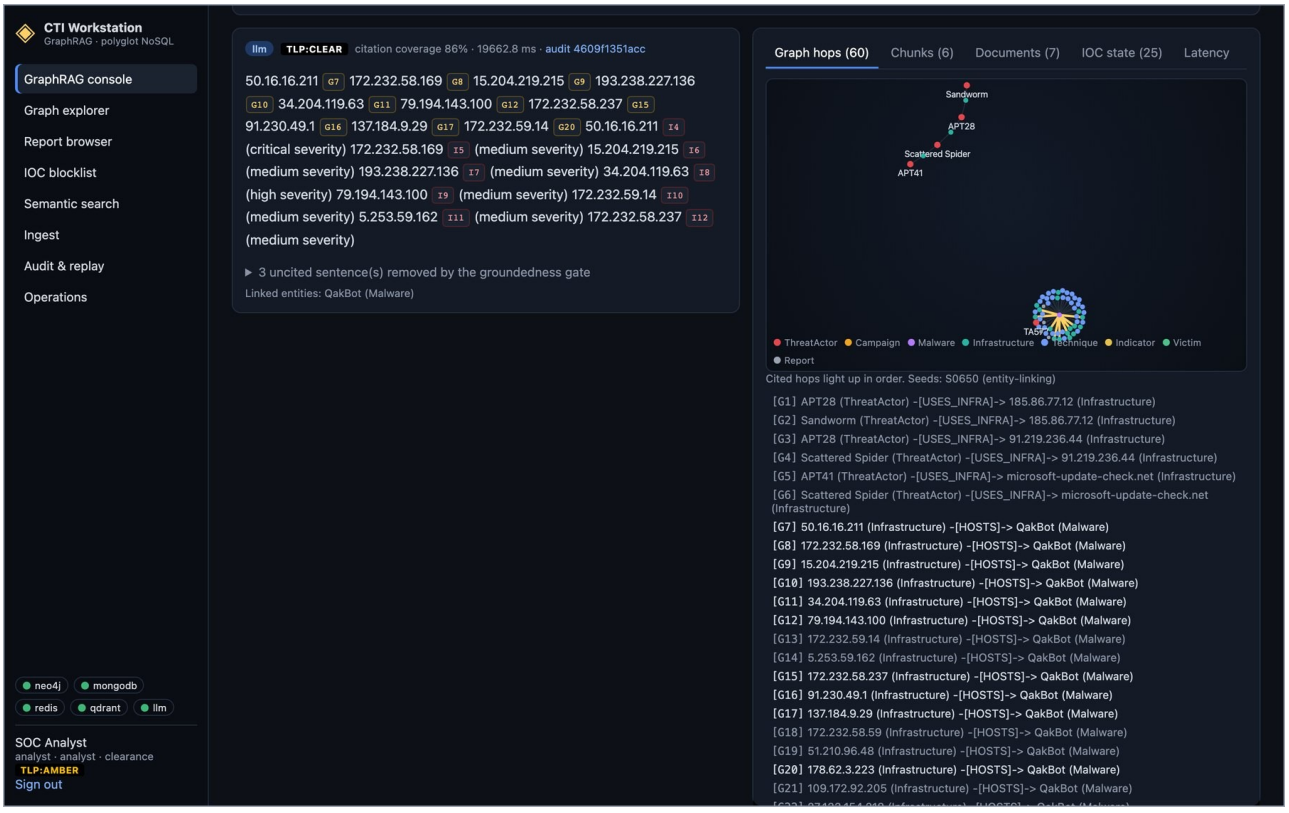


Figure 4. Evidence panel, Graph hops tab: the 60 retrieved edges drawn as a graph and listed. Cited hops are bold, for example [G7] 50.16.16.211 –[HOSTS]→ QakBot.

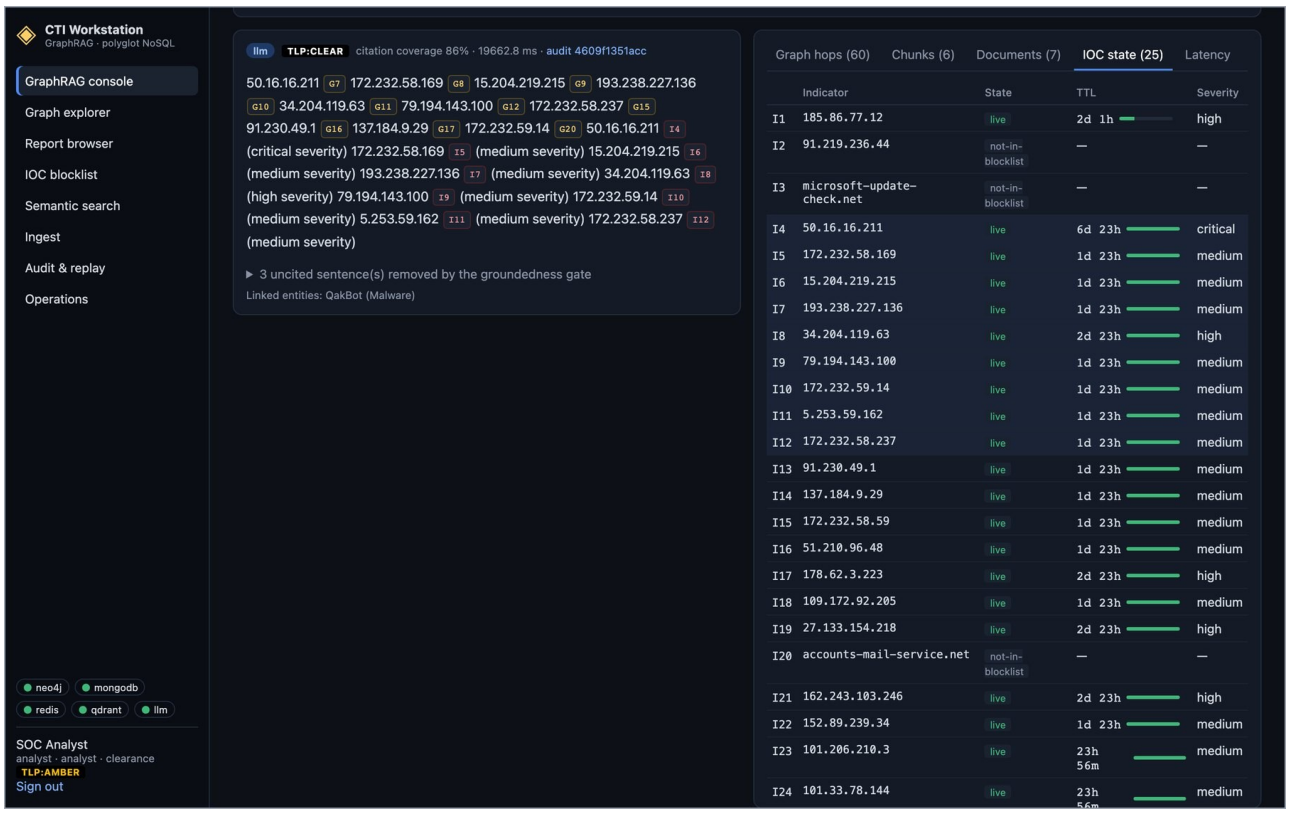


Figure 5. IOC state tab: each indicator in the evidence with its Redis state, remaining TTL and severity. 50.16.16.211 (I4) is live for another 6 d 23 h at critical severity; addresses that are not on the blocklist are marked as such.

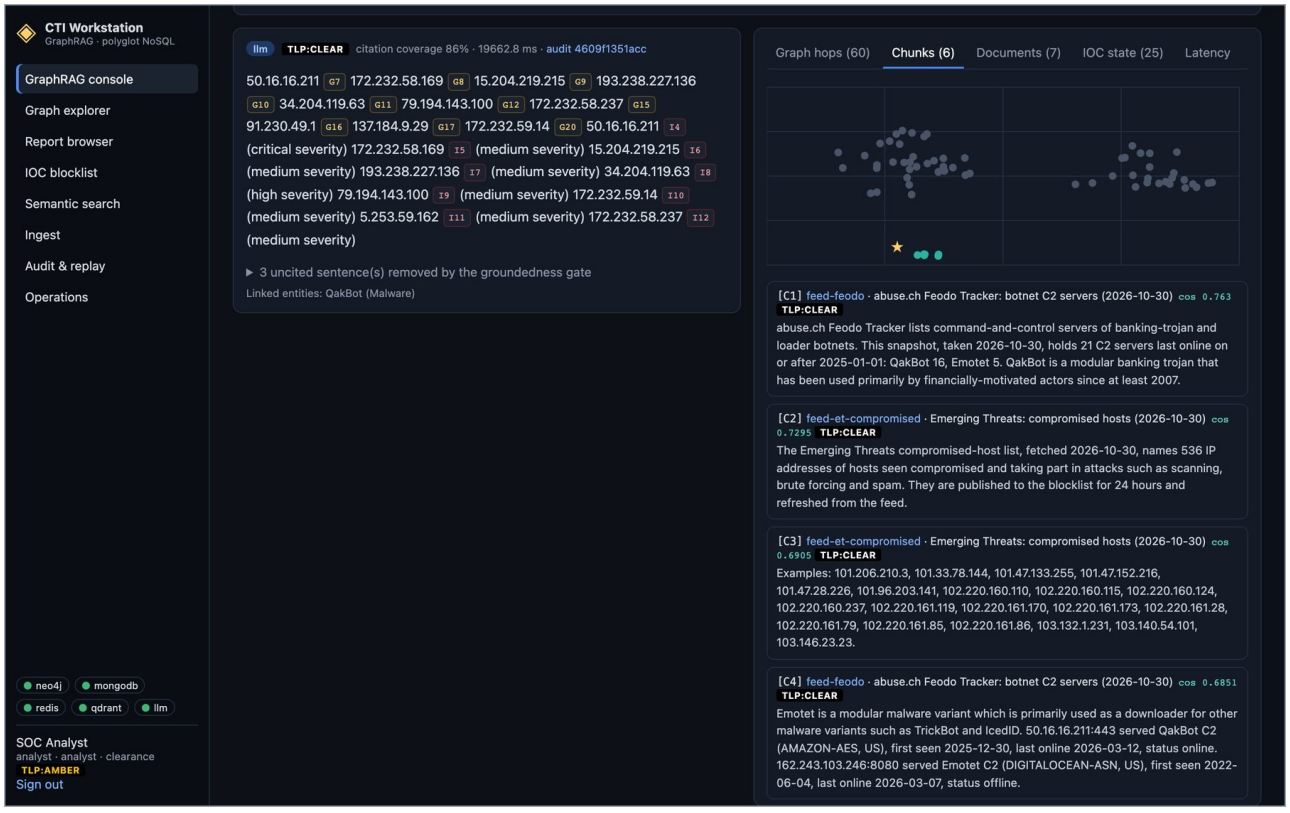


Figure 6. Chunks tab: the passages Qdrant returned for the question, with similarity scores and TLP. C1 and C4 come from today's Feodo Tracker report, C2 and C3 from the Emerging Threats report.

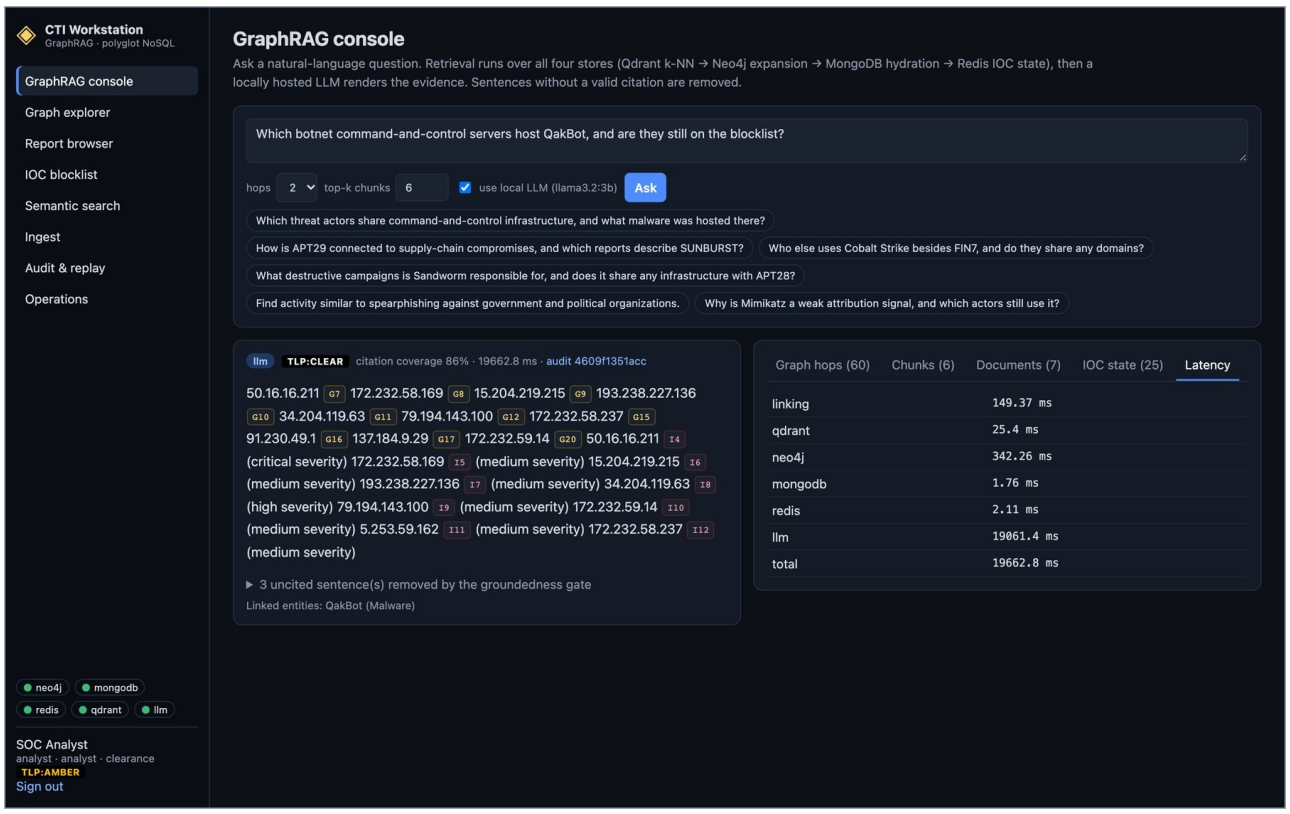


Figure 7. Latency tab: time spent in each step. Retrieval from the four stores takes about half a second in total (Neo4j 342 ms, entity linking 149 ms, Qdrant 25 ms, Redis and MongoDB about 2 ms each). The local model takes 19.1 s.

Figure 8 asks about an ATT&CK campaign. The answer attributes the SolarWinds Compromise to APT29 and names the malware from USES_MALWARE edges. Three yellow cautions appear because Mimikatz, Cobalt Strike and AdFind are each used by many actors (51, 32 and 12): such tools are weak evidence for attribution. These cautions come from node degree in the graph, not from the model.

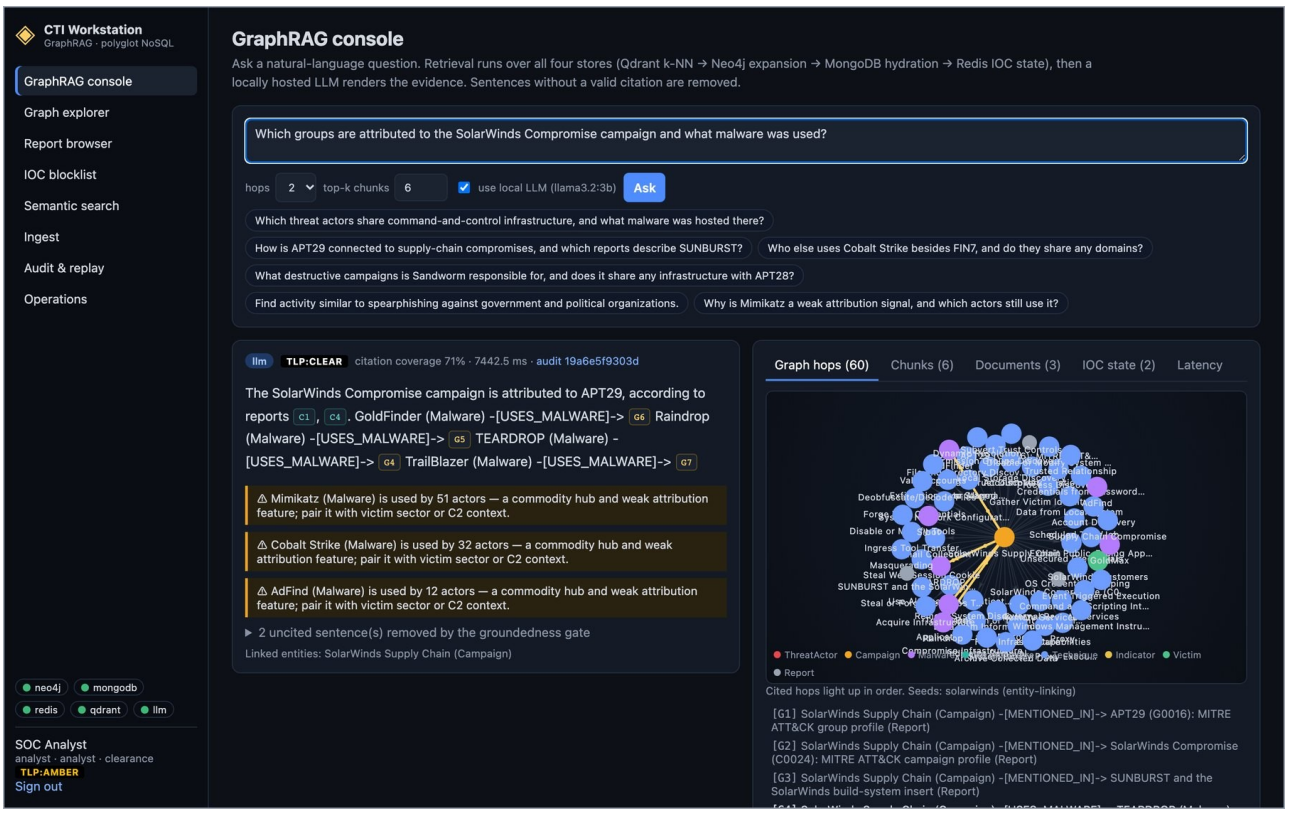


Figure 8. GraphRAG answer for the SolarWinds Compromise (ATT&CK C0024): APT29 with GoldFinder, Raindrop, TEARDROP and TrailBlazer; hub cautions for commodity tools; 71% coverage, 7.4 s.

3.3 Graph explorer

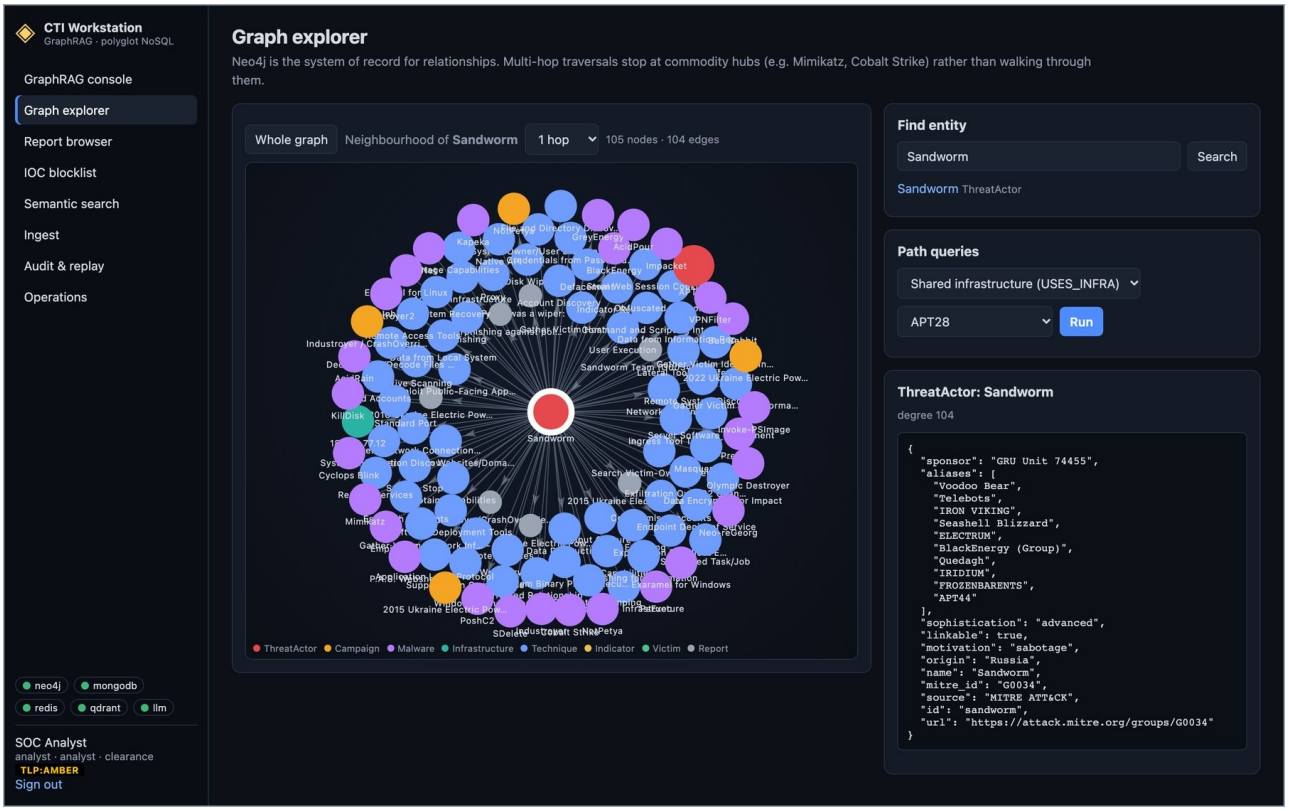


Figure 9. Graph explorer: the 1-hop neighbourhood of Sandworm (105 nodes, 104 edges) loaded from Neo4j, with the node's ATT&CK properties and the path-query panel.

3.4 Report browser

Reports live in MongoDB. Their core fields are shared, but nested fields differ by source: feed metadata, ATT&CK identifiers, or STIX objects. The list shows today's two feed reports first.

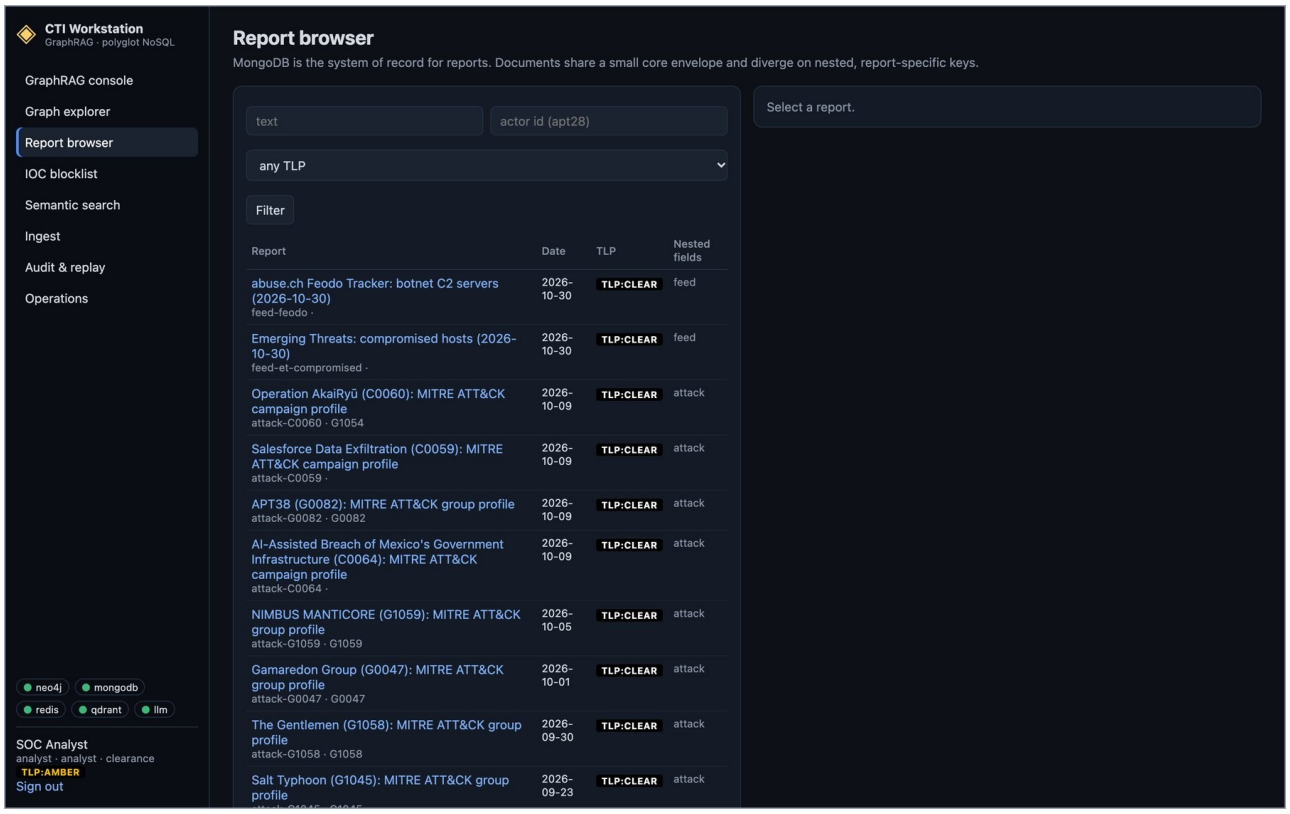


Figure 10. Report browser: newest first, today's Feodo Tracker and Emerging Threats snapshots followed by ATT&CK campaign and group profiles, each with its TLP and nested-field type.

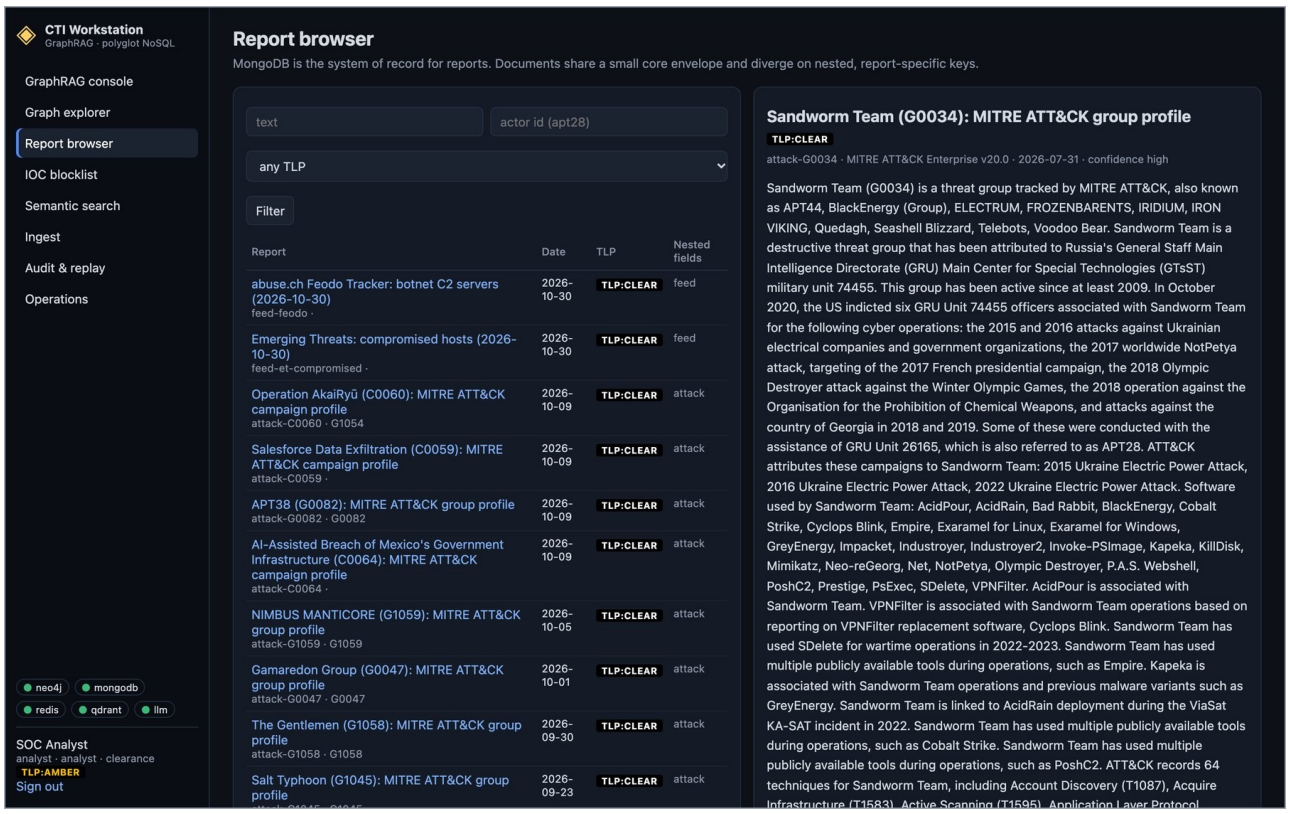


Figure 11. A report opened: the MITRE ATT&CK profile of Sandworm Team (G0034), TLP:CLEAR, read from MongoDB.

3.5 Semantic search

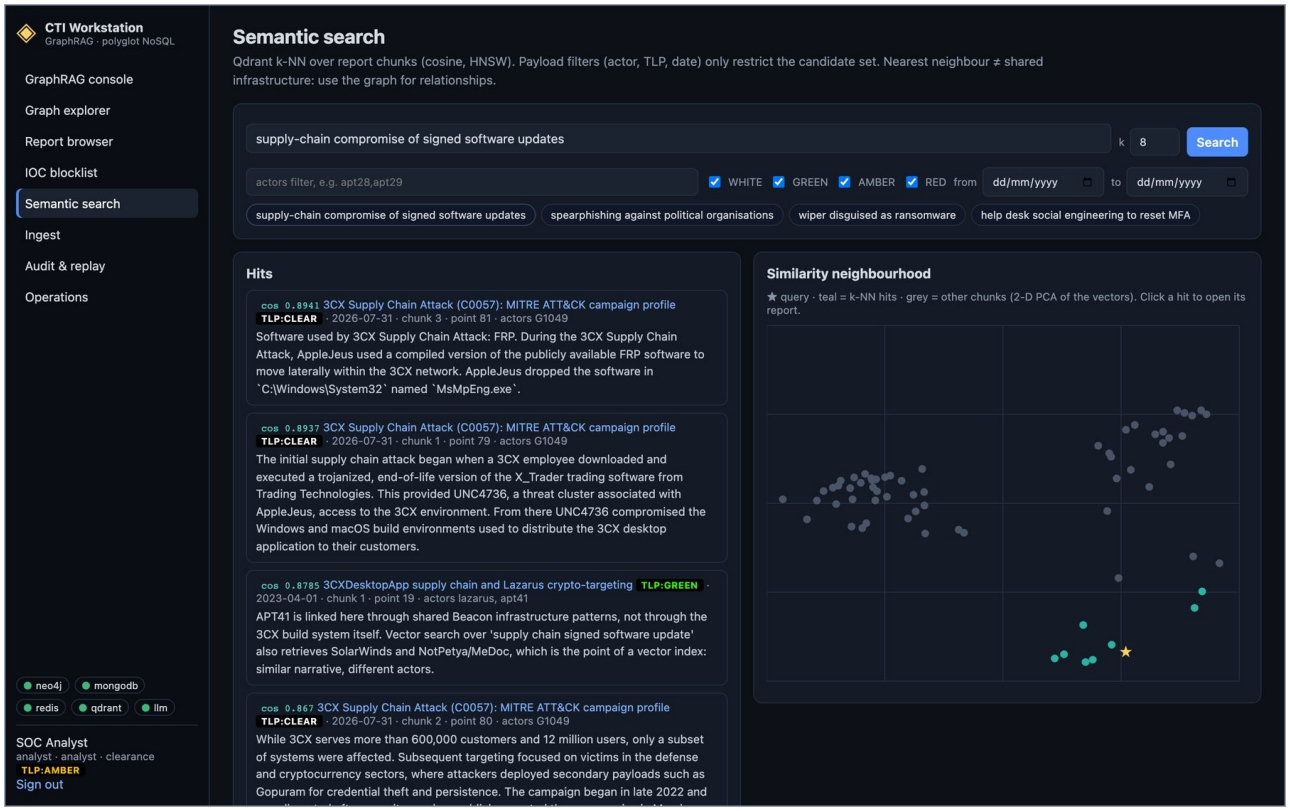


Figure 12. Semantic search for "supply-chain compromise of signed software updates": Qdrant k-NN hits led by the 3CX Supply Chain Attack (C0057), with TLP filters and a 2-D map of the neighbourhood.

3.6 IOC blocklist and the IP gate

The blocklist page reads the Redis hot set and refreshes every 5 seconds. Each key's TTL counts down live, and a key that reaches zero leaves the set by itself. The page lists up to 500 live keys; Redis held 565. The API also refuses any request whose client address is live on the blocklist. Figure 14 shows the two requests refused in Section 4, which MongoDB logged with an automatic expiry.

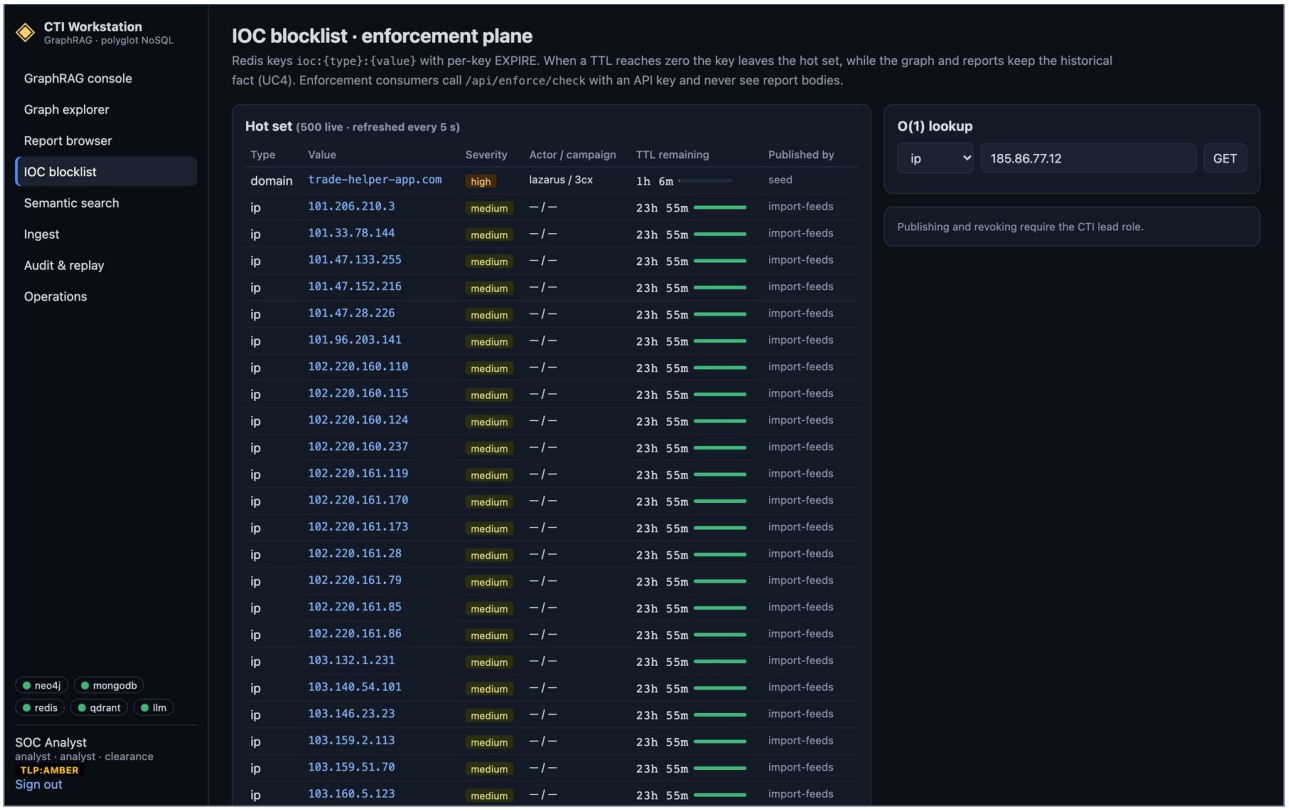


Figure 13. IOC blocklist: live keys from Redis with type, value, severity, source and remaining TTL, plus the O(1) lookup form.

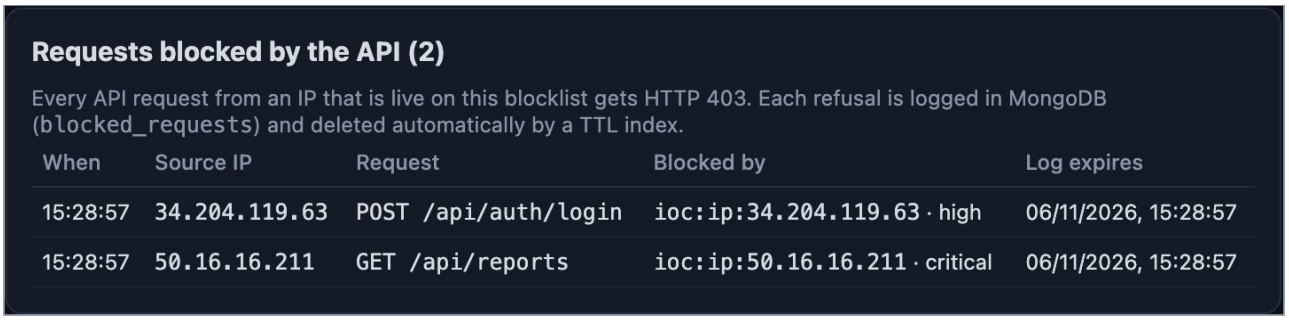


Figure 14. Requests blocked by the API: GET /api/reports from QakBot C2 50.16.16.211 (critical) and POST /api/auth/login from 34.204.119.63 (high), both refused with HTTP 403. Times are UTC; each log entry expires on 6 November 2026.

3.7 Ingest

Signed in as the CTI lead, the sample STIX 2.1 bundle was uploaded. One job writes to the four stores in a fixed order and records each step, so a failed job is retried from the first incomplete step.

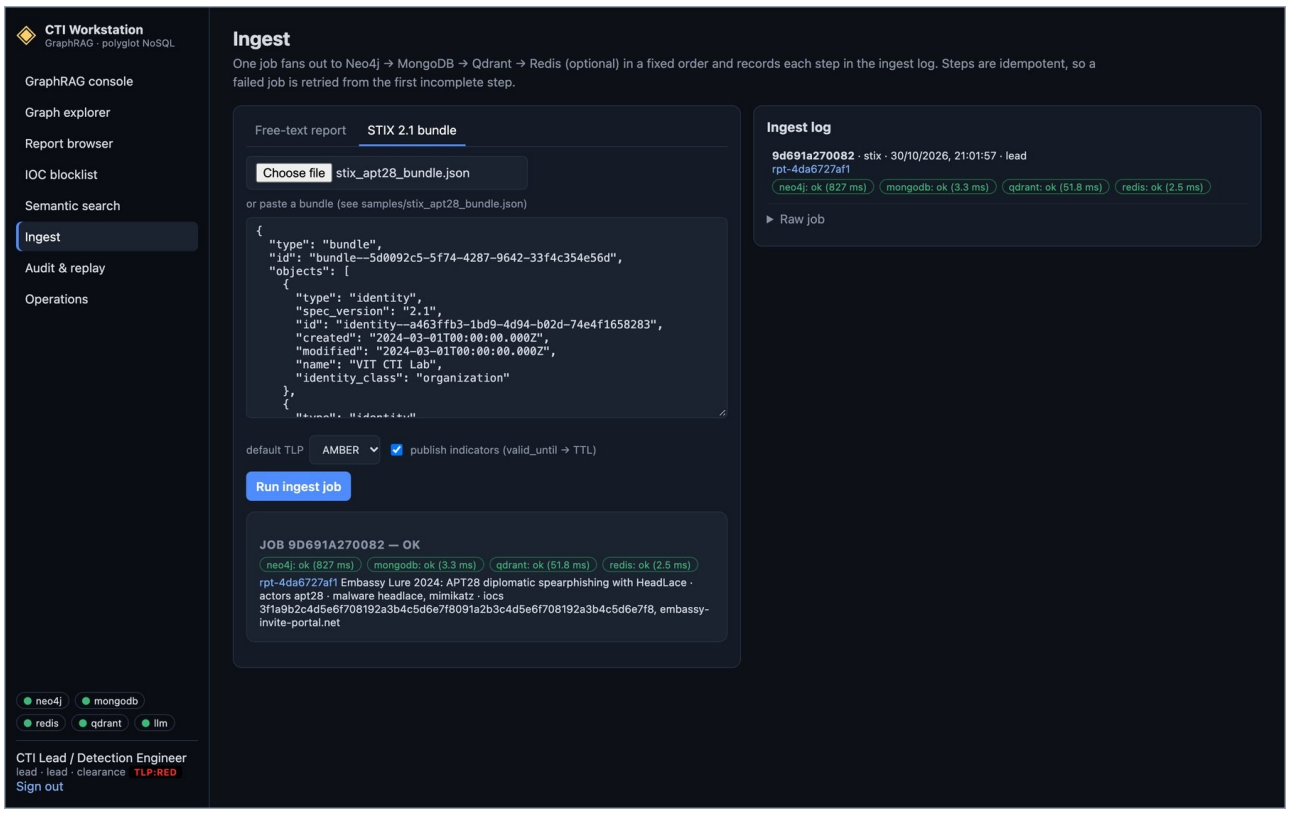


Figure 15. STIX 2.1 ingest by the CTI lead (TLP:RED): job 9d691a270082 succeeded in Neo4j (827 ms), MongoDB (3.3 ms), Qdrant (51.8 ms) and Redis (2.5 ms), creating report rpt-4da6727af1 with its actors, malware and IOCs.

3.8 TLP access control

A junior analyst has TLP:GREEN clearance. The API filters the report list to what that clearance allows. A direct request for a report above it returns the document's existence but withholds the body.

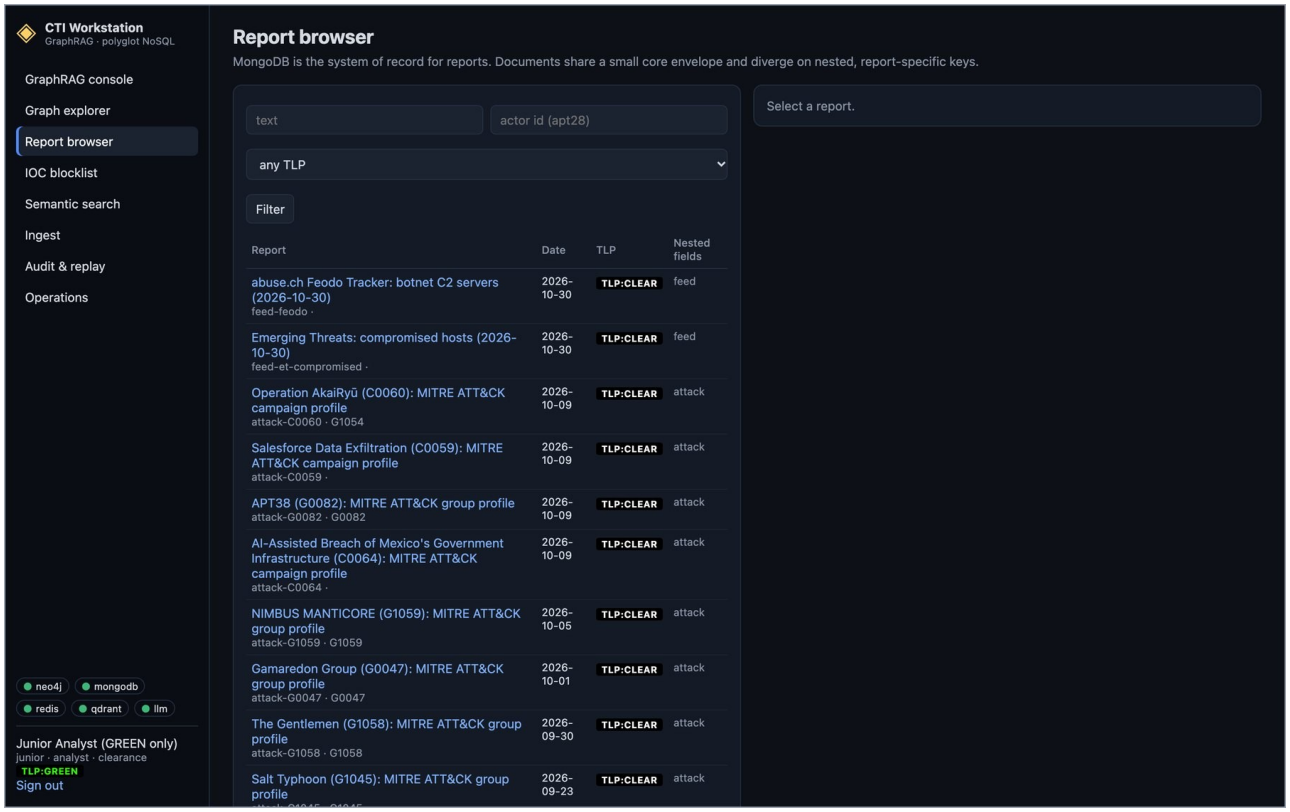


Figure 16. Report browser as the junior analyst (TLP:GREEN): only reports at or below GREEN are listed.

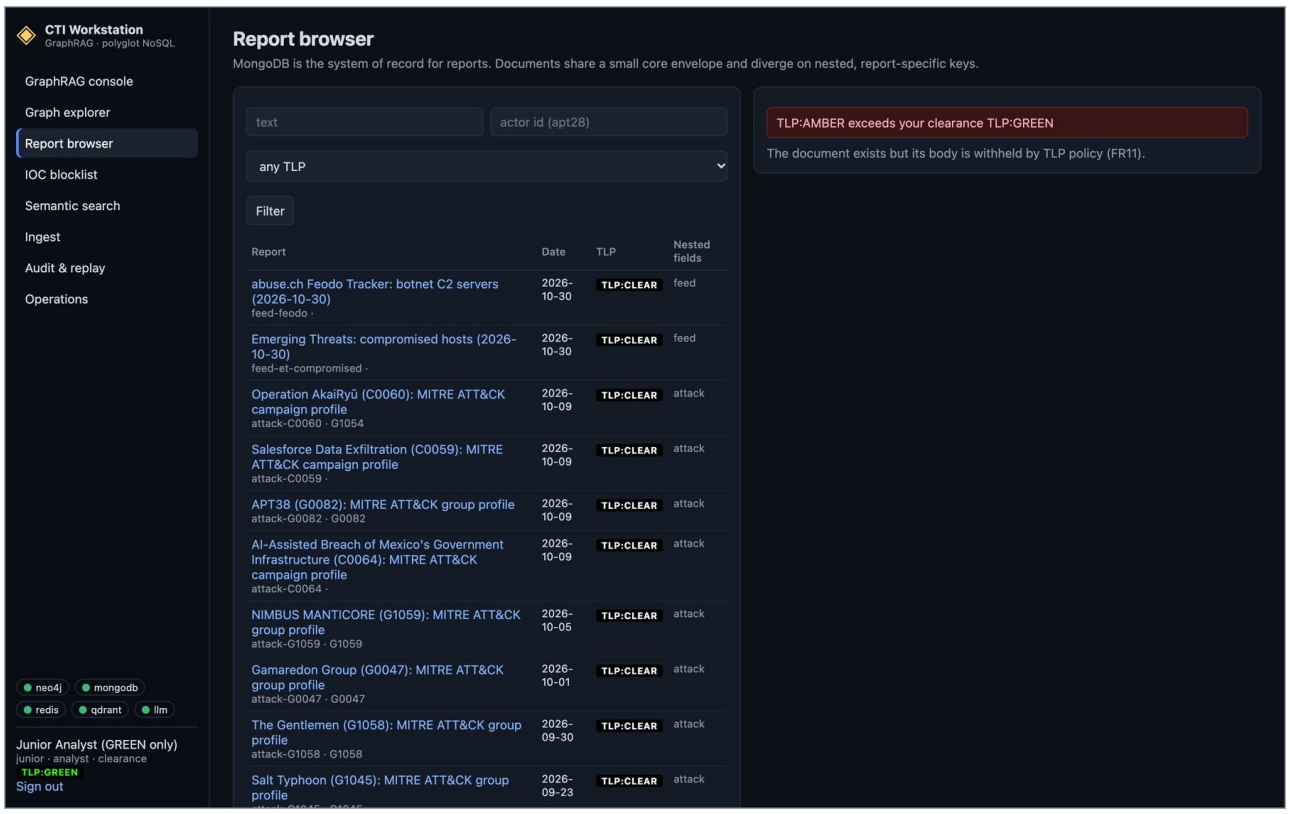


Figure 17. The junior analyst opens rpt-001 (TLP:AMBER) by its address: "TLP:AMBER exceeds your clearance TLP:GREEN" and the body is withheld.

3.9 Audit and replay

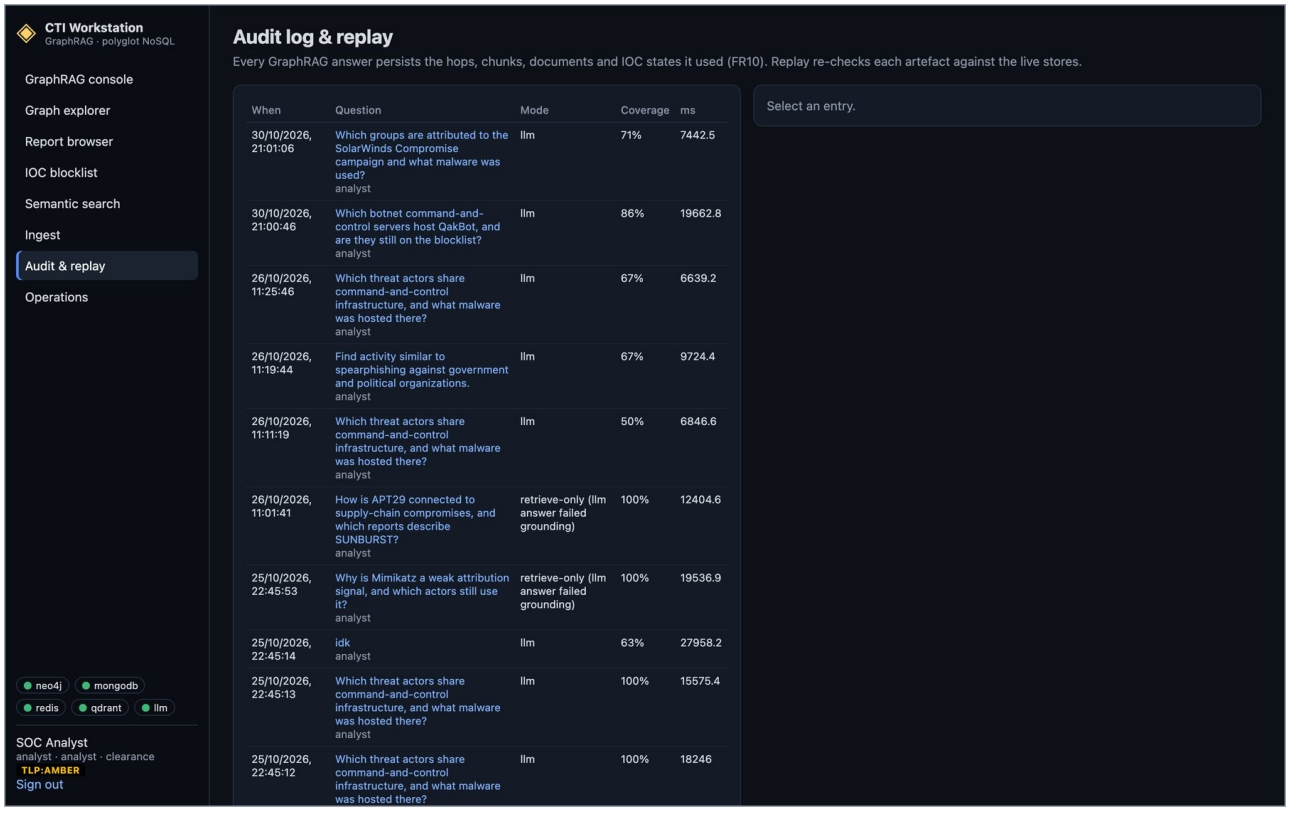


Figure 18. Audit log: every GraphRAG answer with its user, mode, citation coverage and latency, stored in MongoDB. Today's two questions are at the top.

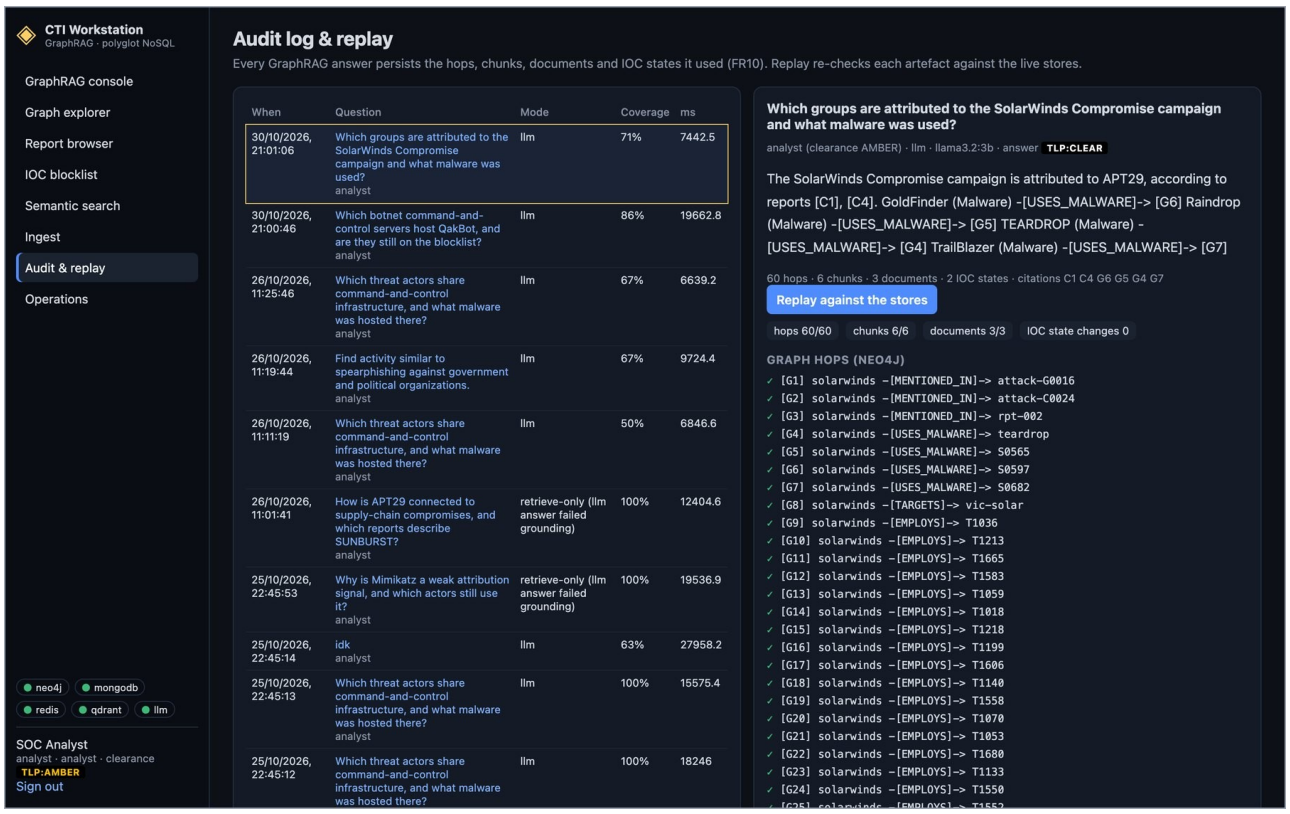


Figure 19. Replay of the SolarWinds answer against the live stores: all 60 hops, 6 chunks and 3 documents still exist and no IOC state has changed, so the answer can be reproduced.

3.10 Operations

The Operations page shows store liveness, latency against the service-level objectives (SLOs), the graph inventory and the cross-store integrity check. Its latency table covers only requests that this API process has served since it started: 2 GraphRAG answers and 30 Redis lookups. With so few samples, the first answer (19.7 s) and the first Redis calls set the high percentiles, and two cells are red. The controlled evaluation in Section 7 sends more requests and meets both SLOs: Redis p99 1.27 ms against 5 ms, and GraphRAG p95 10.6 s against 15 s.

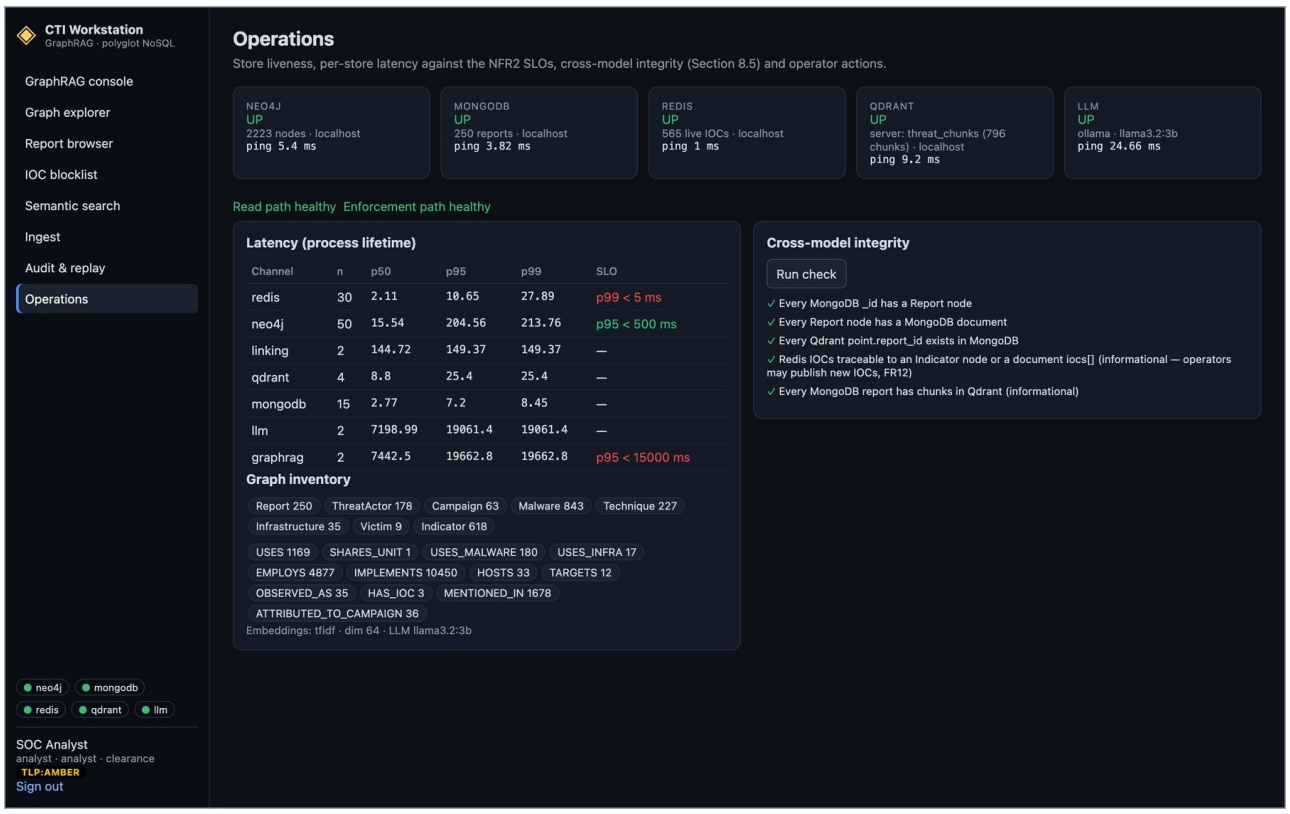


Figure 20. Operations: all four stores and the model UP; read and enforcement paths healthy; per-channel latency since process start; graph inventory; and all five cross-model integrity rules passing.

4. Working System: REST and Enforcement API

The console is one client of the API. The service exposes 36 endpoints and documents them with OpenAPI. Analysts authenticate with a JWT. Firewall, EDR and SIEM consumers use an API key, and their path reads only Redis.

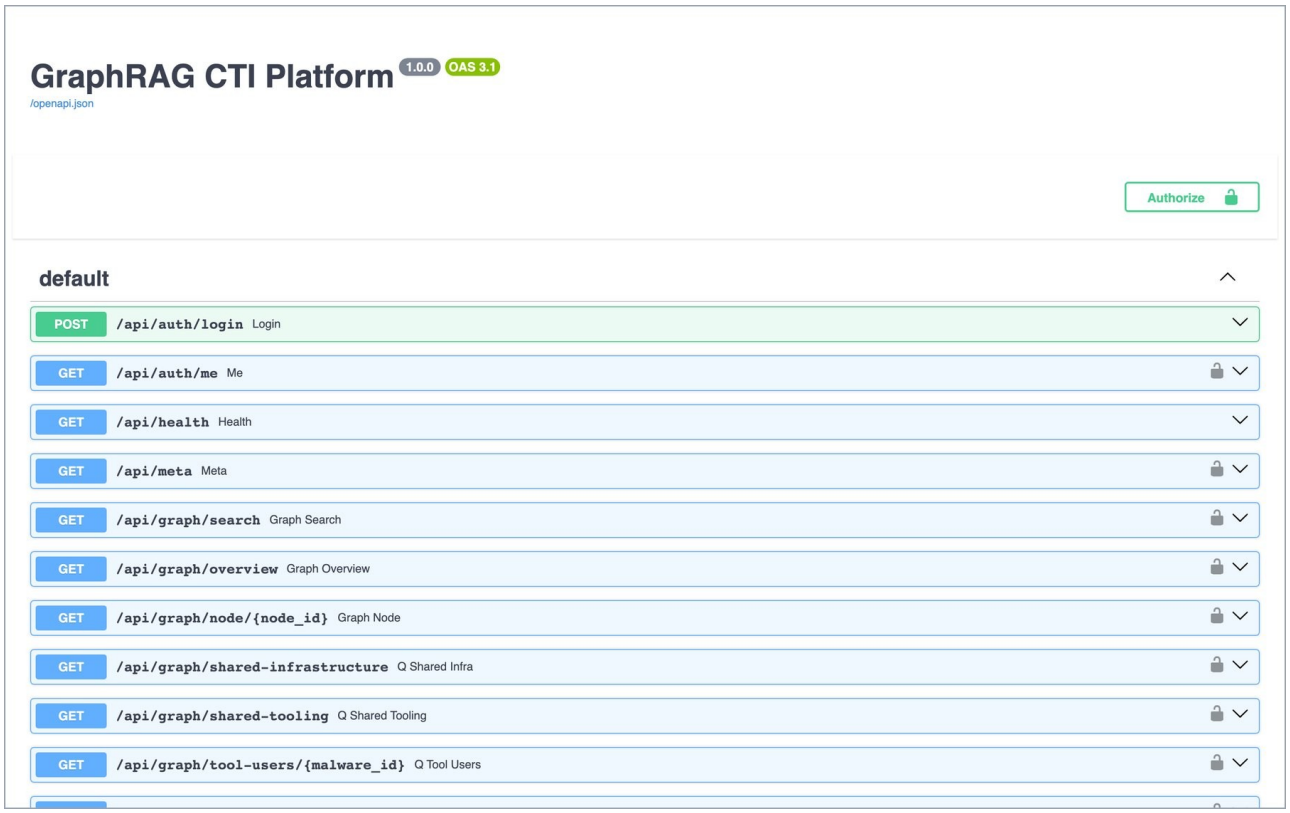


Figure 21. Interactive API documentation served by the running service (FastAPI, OpenAPI 3.1): 36 endpoints for auth, graph, reports, blocklist, enforcement, search, GraphRAG, audit, ingest and admin.

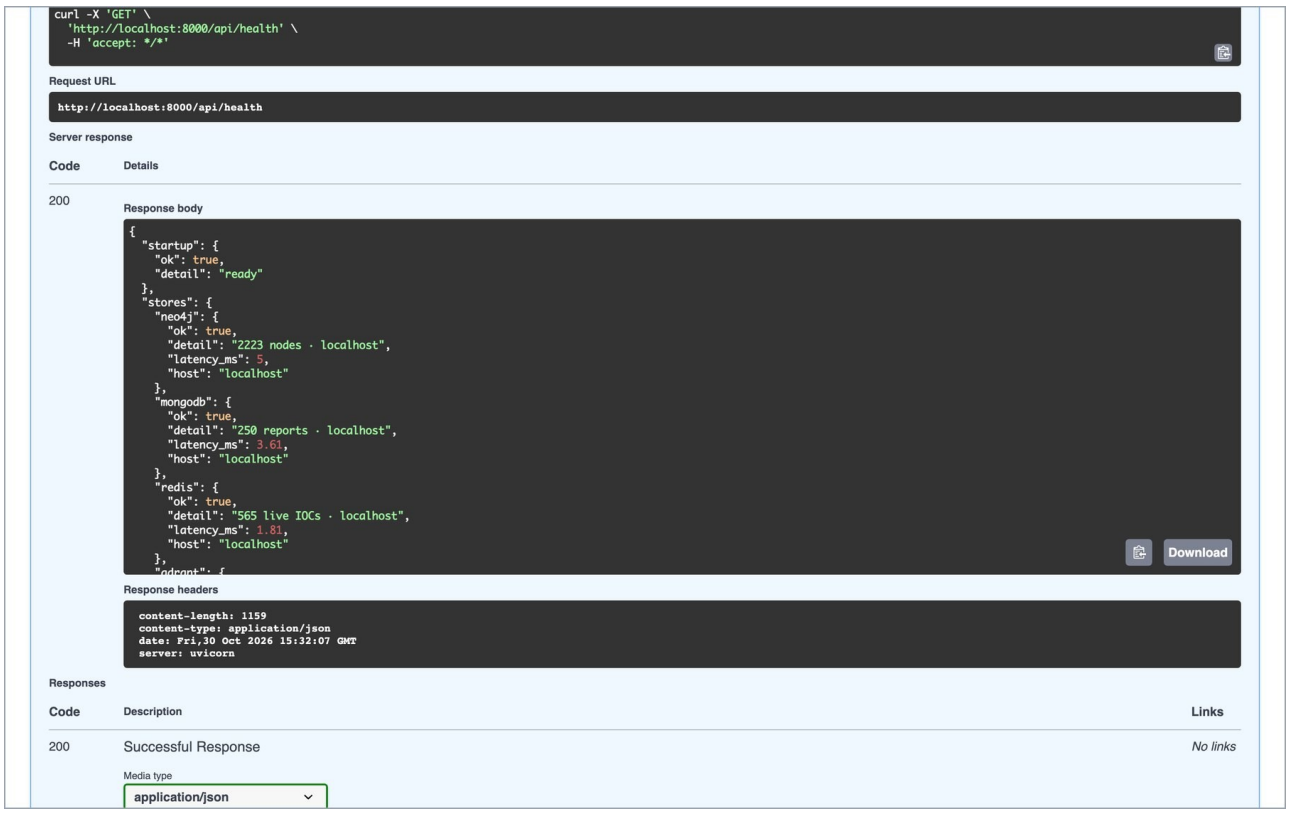


Figure 22. GET /api/health executed from the documentation page: HTTP 200 with each store's status, contents and ping time.

4.1 Enforcement lookups and the IP gate

These commands were run against the live API. Check 1 is the lookup a firewall makes before it lets traffic through to an address. Checks 3 and 4 are the IP gate: the same request is refused when it comes from a C2 server on the blocklist and passes on to authentication when it comes from an ordinary address. If Redis is unreachable, the enforcement endpoint fails closed and answers blocked with HTTP 503.

```
# 1. firewall / EDR enforcement lookup (Redis only) for a QakBot C2 server from abuse.ch
    Feodo Tracker
$ curl -s -H "X-API-Key: $CONSUMER_API_KEY" "localhost:8000/api/enforce/check?type=ip&
    value=50.16.16.211"
{"blocked":true,"ttl_remaining":604698,"severity":"critical"}

# 2. an address that is not on the blocklist
$ curl -s -H "X-API-Key: $CONSUMER_API_KEY" "localhost:8000/api/enforce/check?type=ip&
    value=8.8.8.8"
{"blocked":false,"ttl_remaining":0,"severity":null}

# 3. the IP gate: an API request arriving from that C2 address is refused
$ curl -s -w "  HTTP %{http_code}\n" -H "X-Forwarded-For: 50.16.16.211"
    localhost:8000/api/reports
{"detail":"50.16.16.211 is on the IOC blocklist","blocked":true,"ttl_remaining":604698,
    "severity":"critical"}  HTTP 403
$ curl -s -w "  HTTP %{http_code}\n" -H "X-Forwarded-For: 34.204.119.63" -X POST
    localhost:8000/api/auth/login -d "{}"
{"detail":"34.204.119.63 is on the IOC blocklist","blocked":true,"ttl_remaining":259098,
    "severity":"high"}  HTTP 403

# 4. the same request from an ordinary address reaches the API (401: no token yet)
$ curl -s -w "  HTTP %{http_code}\n" -H "X-Forwarded-For: 8.8.8.8"
    localhost:8000/api/reports
{"detail":"authentication required"}  HTTP 401
```

5. Live Databases

This section shows the data inside each database, using that database's own console and command-line tool rather than the platform. Every count matches the corresponding view in the console (Section 3).

5.1 Neo4j: the threat knowledge graph

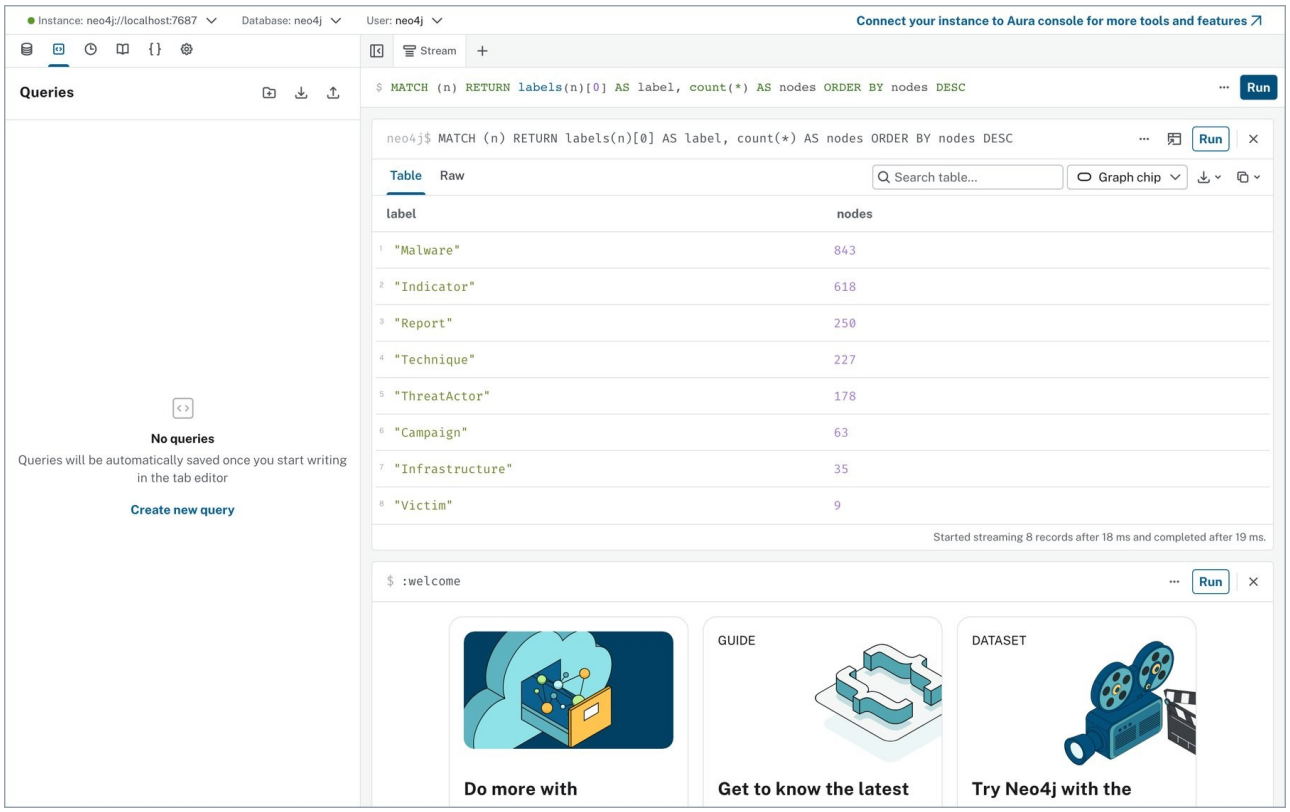


Figure 23. Neo4j Browser connected to the local instance: node count per label (Malware 843, Indicator 618, Report 250, Technique 227, ThreatActor 178, Campaign 63, Infrastructure 35, Victim 9).

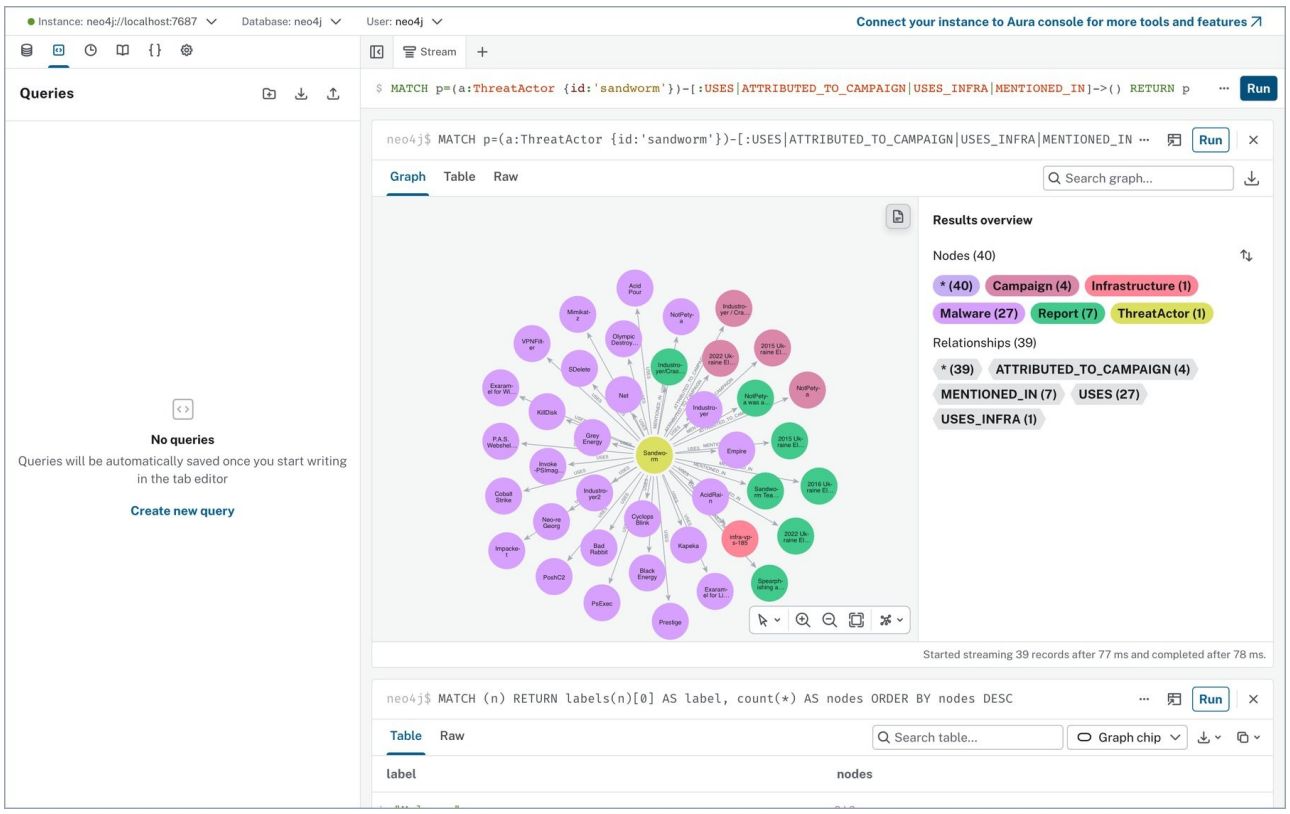


Figure 24. Cypher: Sandworm's malware, campaigns, infrastructure and reports (40 nodes, 39 relationships).

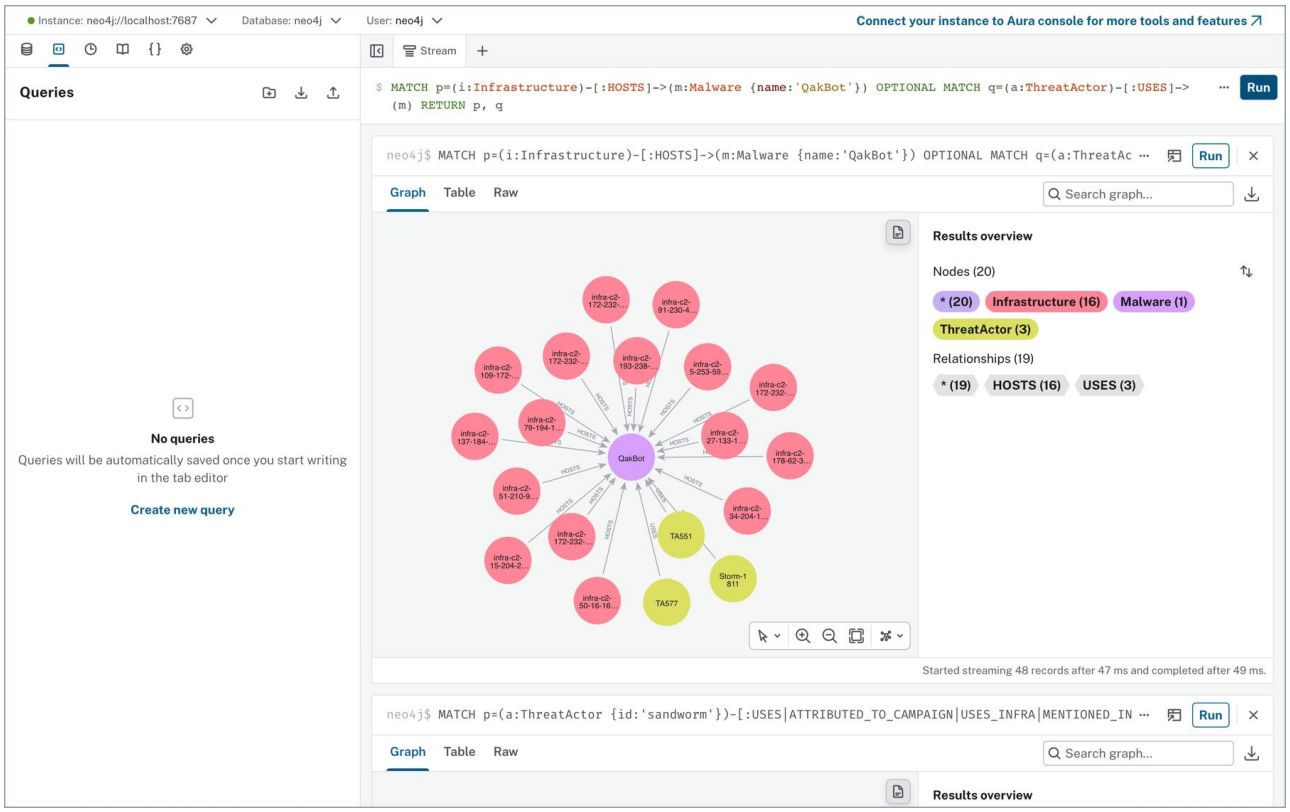


Figure 25. Cypher: the 16 Feodo Tracker C2 servers that HOST QakBot and the three threat actors that USE it (20 nodes, 19 relationships).

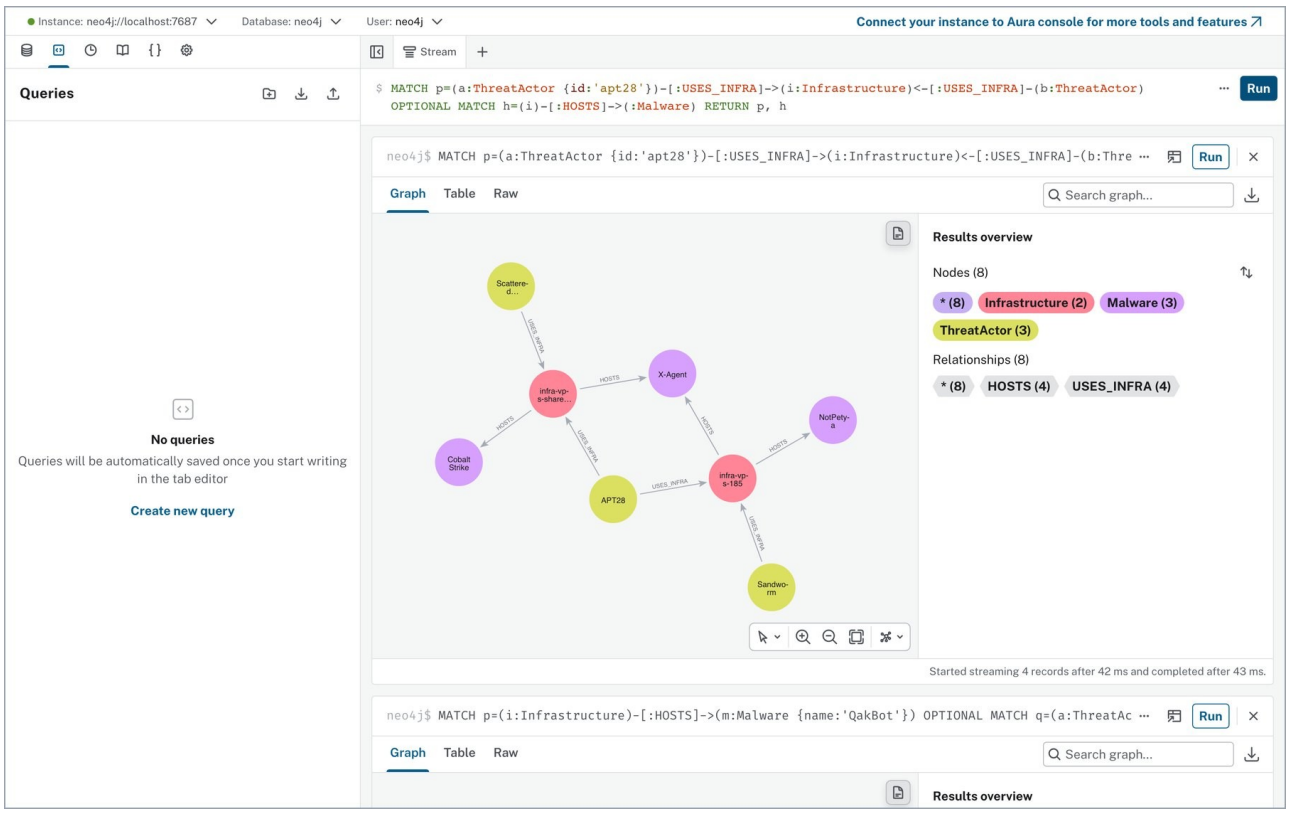


Figure 26. Cypher: infrastructure that APT28 shares with other actors, and the malware it hosts. This is the multi-hop query behind the shared-infrastructure question.

cypher-shell inside the Neo4j container

```
$ docker exec cti-neo4j cypher-shell -u neo4j -p *** "MATCH (n) RETURN labels(n)[0] AS
    label, count(*) AS nodes ORDER BY nodes DESC"
label, nodes
"Malware", 843
"Indicator", 618
"Report", 250
"Technique", 227
"ThreatActor", 178
"Campaign", 63
"Infrastructure", 35
"Victim", 9

$ docker exec cti-neo4j cypher-shell -u neo4j -p *** "MATCH ()-[r]->() RETURN count(r) AS
    relationships"
relationships
18491

$ docker exec cti-neo4j cypher-shell -u neo4j -p *** "CALL dbms.components() YIELD name,
    versions, edition"
name, versions, edition
"Neo4j Kernel", ["5.26.31"], "community"
```

5.2 MongoDB: reports and logs

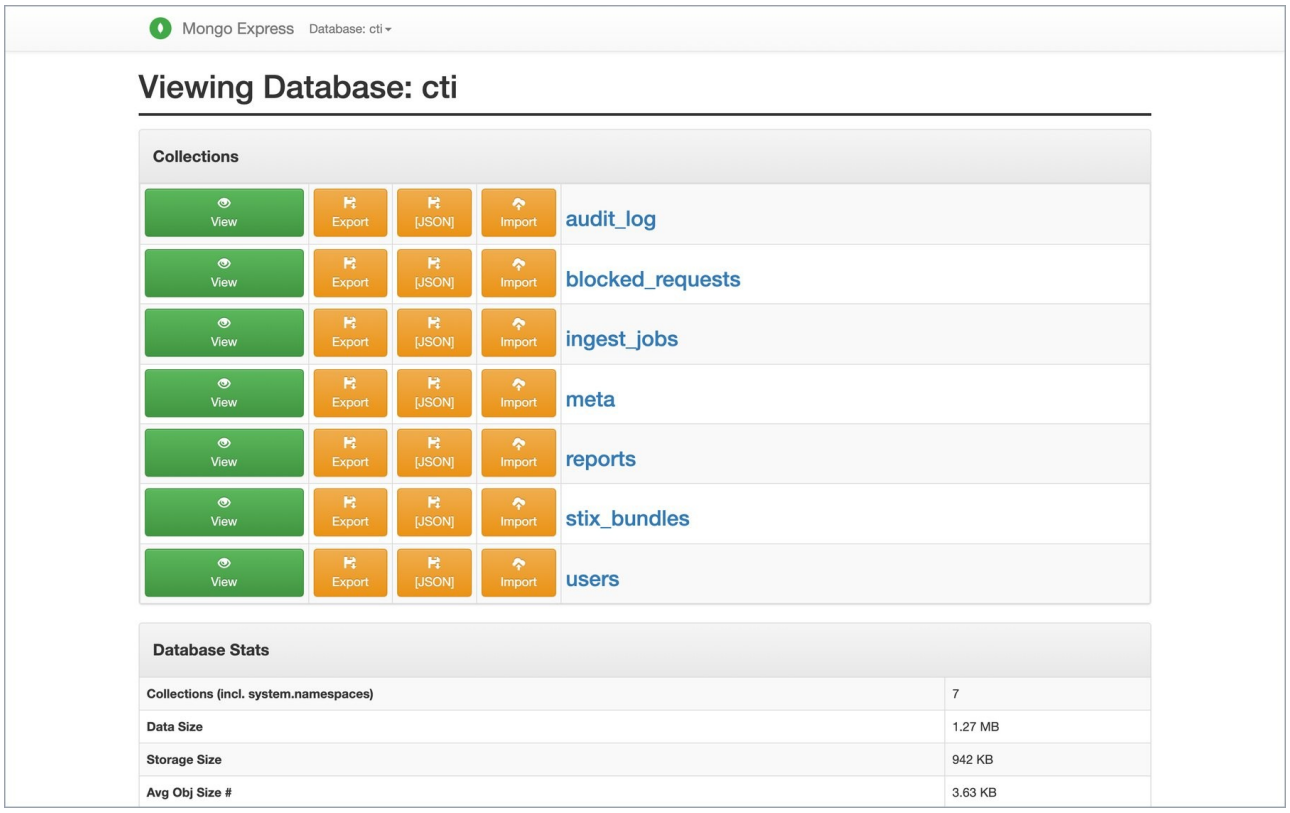


Figure 27. mongo-express: database cti and its seven collections, with data size.

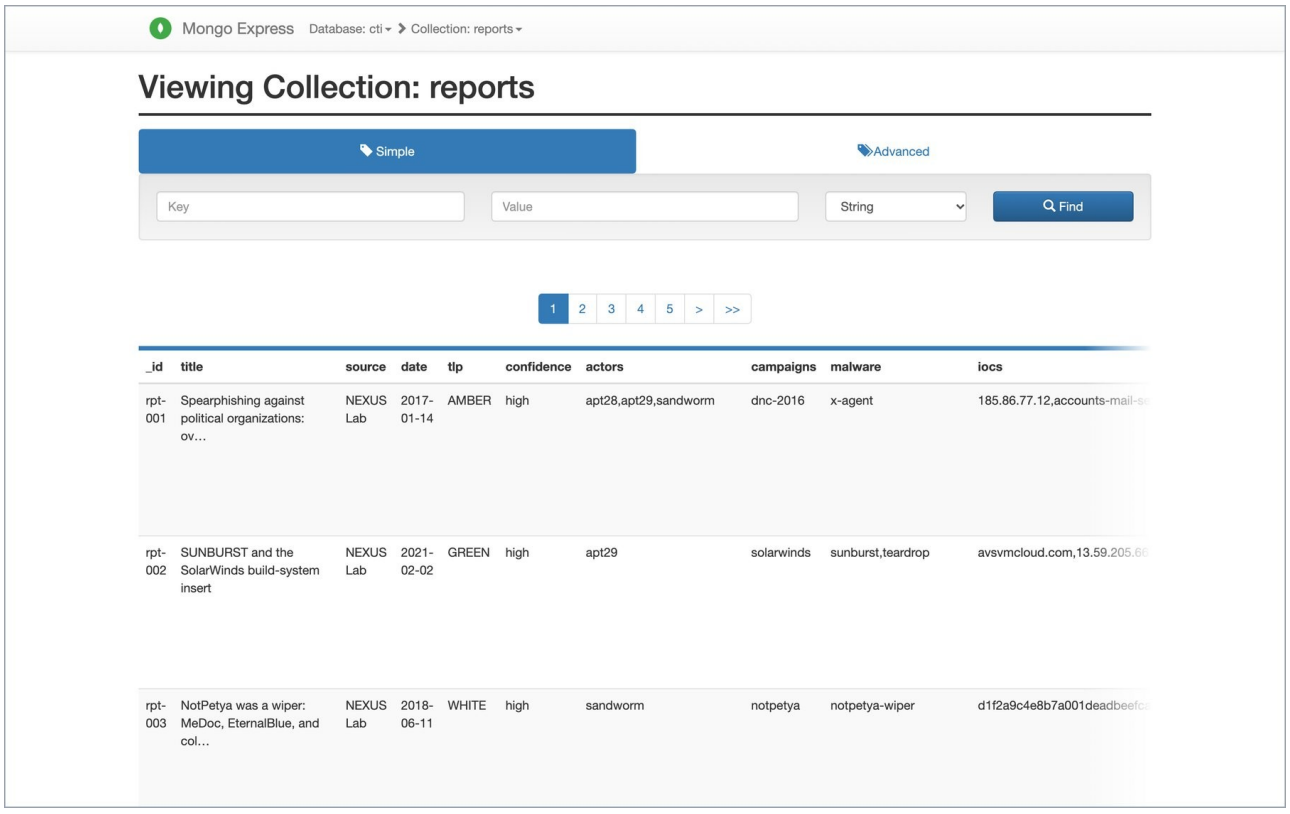


Figure 28. Collection reports. The documents share a core (title, source, date, TLP, actors, IOCs) but carry different nested fields.

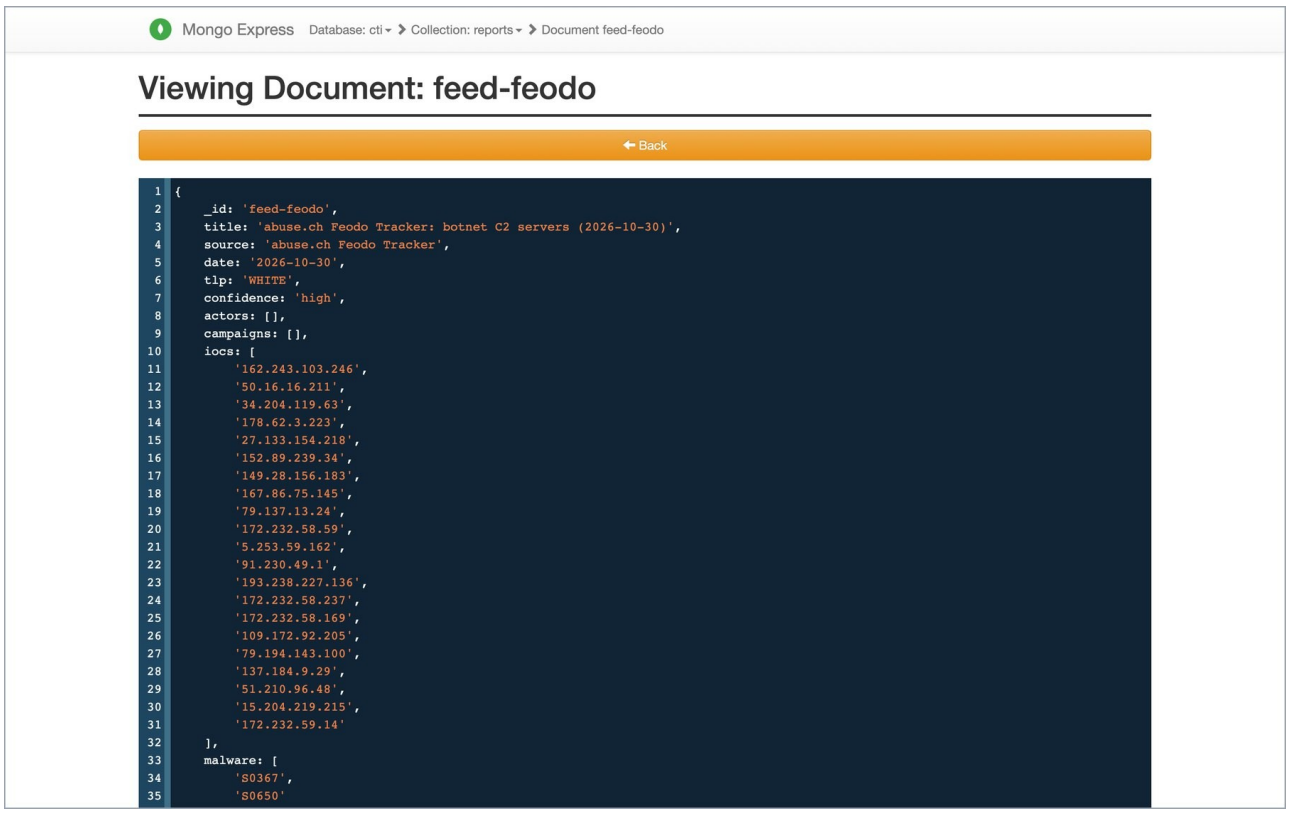


Figure 29. Document feed-feodo: today's abuse.ch Feodo Tracker snapshot, with its 21 C2 addresses in iocs[], the ATT&CK ids of QakBot and Emotet, and the feed URL.

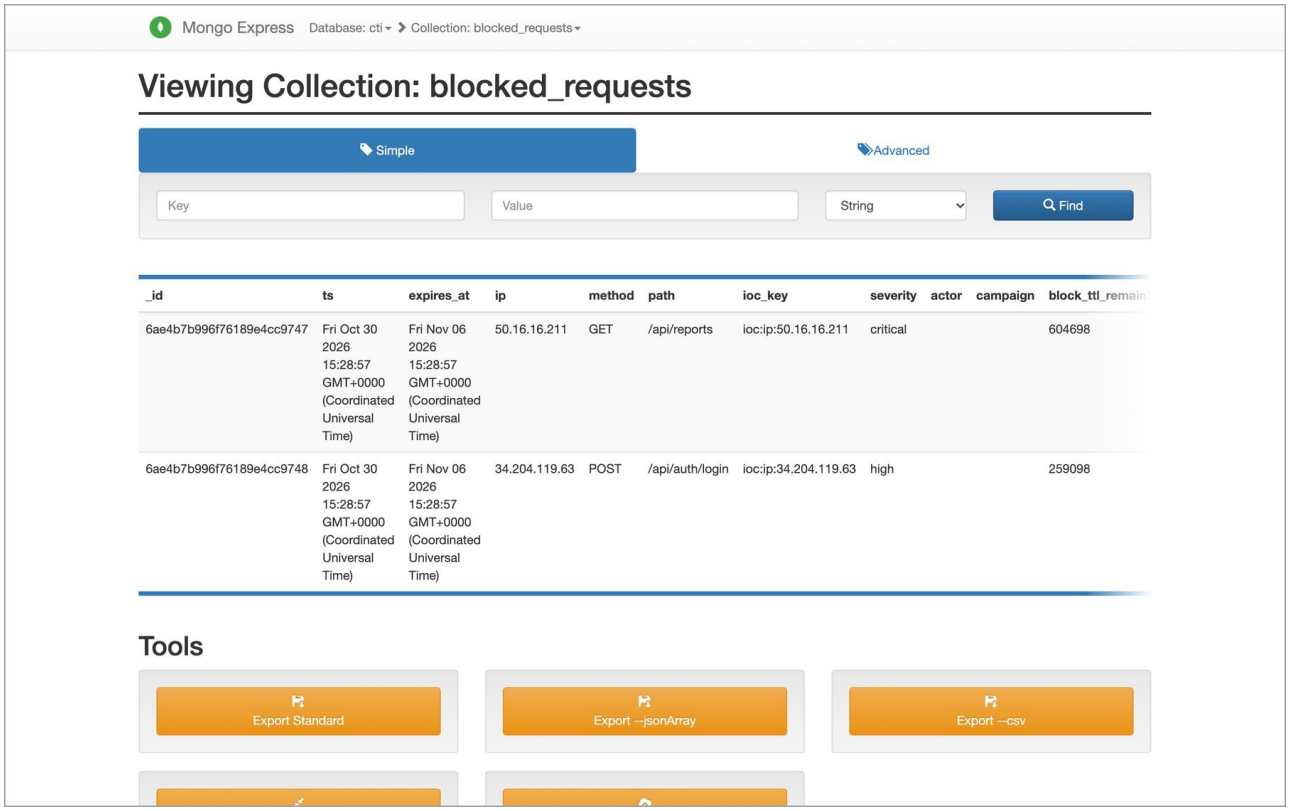


Figure 30. Collection blocked_requests: the two requests refused by the IP gate in Section 4. A TTL index on expires_at deletes each entry after seven days.

mongosh inside the MongoDB container

```
$ docker exec cti-mongo mongosh cti --quiet --eval
    "db.getCollectionNames().sort().forEach(c => print(c.padEnd(18),
    db[c].countDocuments()))"
audit_log          87
blocked_requests   2
ingest_jobs        1
meta               4
reports            250
stix_bundles       1
users              4

$ docker exec cti-mongo mongosh cti --quiet --eval
    'db.reports.aggregate([{$group:{_id:"$source", reports:{$sum:1}}},
    {$sort:{reports:-1}}]).forEach(d => print(String(d._id).padEnd(34), d.reports))'
MITRE ATT&CK Enterprise v20.0      235
NEXUS Lab                          12
abuse.ch Feodo Tracker             1
VIT CTI Lab                        1
Emerging Threats (Proofpoint)      1

$ docker exec cti-mongo mongosh cti --quiet --eval 'print(db.reports.getIndexes().map(i =>
    i.name).join(", "))'
_id_, actors_1, date_-1, tlp_1, iocs_1

$ docker exec cti-mongo mongosh cti --quiet --eval 'db.blocked_requests.find({}, {_id:0,
    ip:1, method:1, path:1, ioc_key:1, expires_at:1}).forEach(d => printjson(d))'
{
  expires_at: ISODate('2026-11-06T15:28:57.979Z'),
  ip: '50.16.16.211',
  method: 'GET',
  path: '/api/reports',
  ioc_key: 'ioc:ip:50.16.16.211'
}
{
  expires_at: ISODate('2026-11-06T15:28:57.998Z'),
  ip: '34.204.119.63',
  method: 'POST',
  path: '/api/auth/login',
  ioc_key: 'ioc:ip:34.204.119.63'
}

$ docker exec cti-mongo mongosh --quiet --eval "db.version()"
7.0.43
```

5.3 Redis: the IOC blocklist

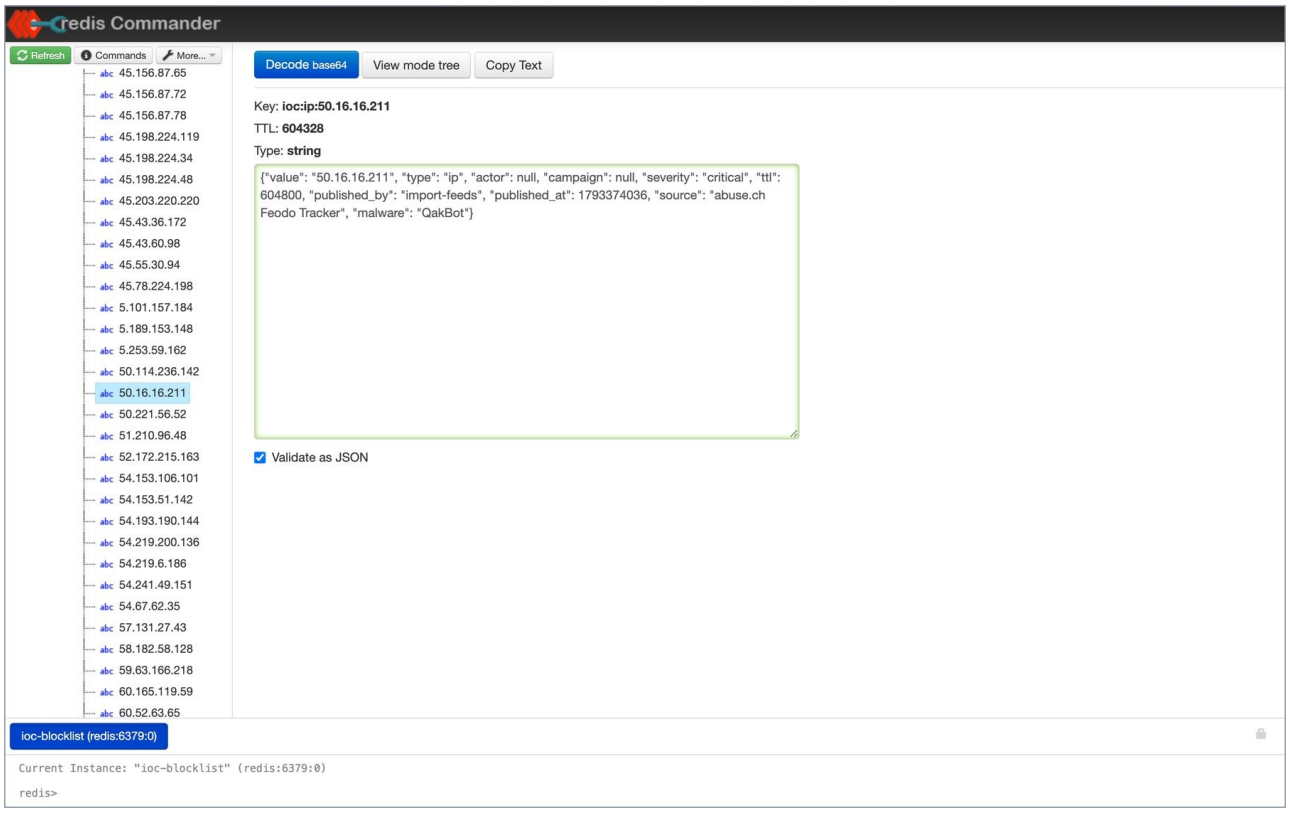


Figure 31. Redis Commander (read-only): key ioc:ip:50.16.16.211 with TTL 604,318 s (about 7 days) and its JSON value: critical, QakBot, published by import-feeds from abuse.ch Feodo Tracker.

The keyspace holds one key per IOC plus the sorted set `ioc:index`, which orders keys by expiry time so the hot set can be listed without scanning. `DBSIZE` is therefore one more than the number of live IOCs.

redis-cli inside the Redis container

```
$ docker exec cti-redis redis-cli INFO server | grep redis_version
redis_version:7.4.11
$ docker exec cti-redis redis-cli DBSIZE
566
$ docker exec cti-redis redis-cli ZCOUNT ioc:index -inf +inf
565
$ docker exec cti-redis redis-cli GET ioc:ip:50.16.16.211
{"value": "50.16.16.211", "type": "ip", "actor": null, "campaign": null, "severity":
    "critical", "ttl": 604800, "published_by": "import-feeds", "published_at": 1793374036,
    "source": "abuse.ch Feodo Tracker", "malware": "QakBot"}
$ docker exec cti-redis redis-cli TTL ioc:ip:50.16.16.211
604227
$ docker exec cti-redis redis-cli GET ioc:ip:34.204.119.63
{"value": "34.204.119.63", "type": "ip", "actor": null, "campaign": null, "severity":
    "high", "ttl": 259200, "published_by": "import-feeds", "published_at": 1793374036,
    "source": "abuse.ch Feodo Tracker", "malware": "QakBot"}
$ docker exec cti-redis redis-cli TTL ioc:ip:34.204.119.63
258627
$ docker exec cti-redis redis-cli EXISTS ioc:ip:8.8.8.8
0
```

5.4 Qdrant: report embeddings

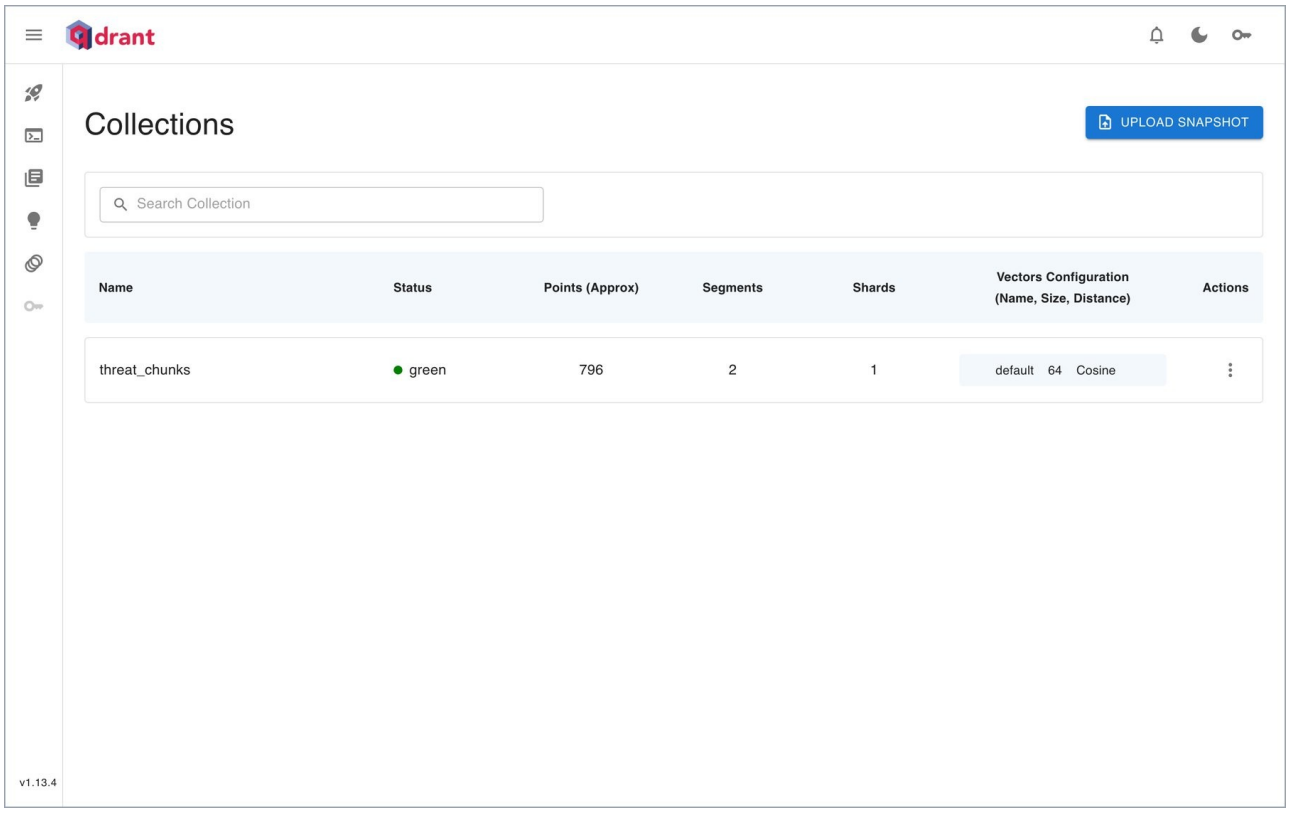


Figure 32. Qdrant dashboard: collection threat_chunks, status green, 796 points, 64-dimensional vectors with cosine distance.

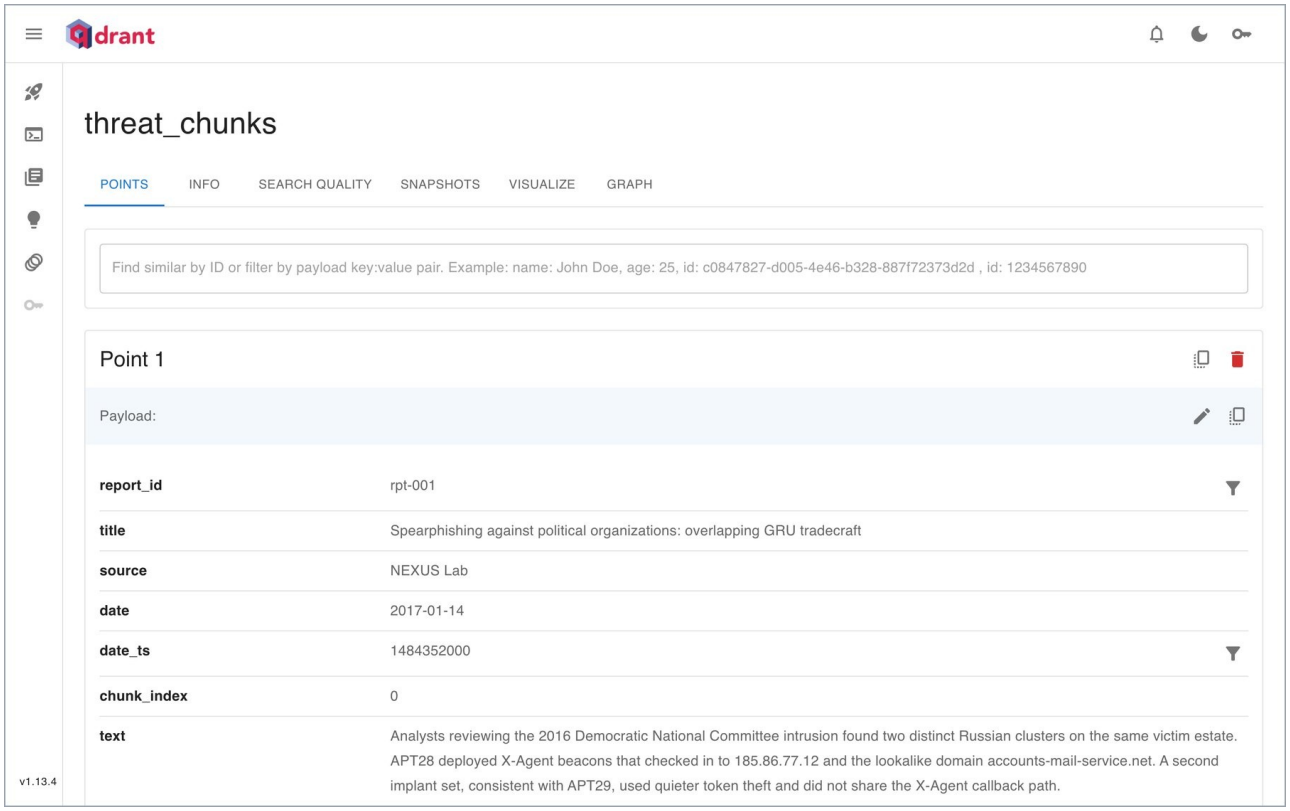


Figure 33. A point and its payload: report_id, title, source, date, chunk index and the chunk text. Search filters on these payload fields.

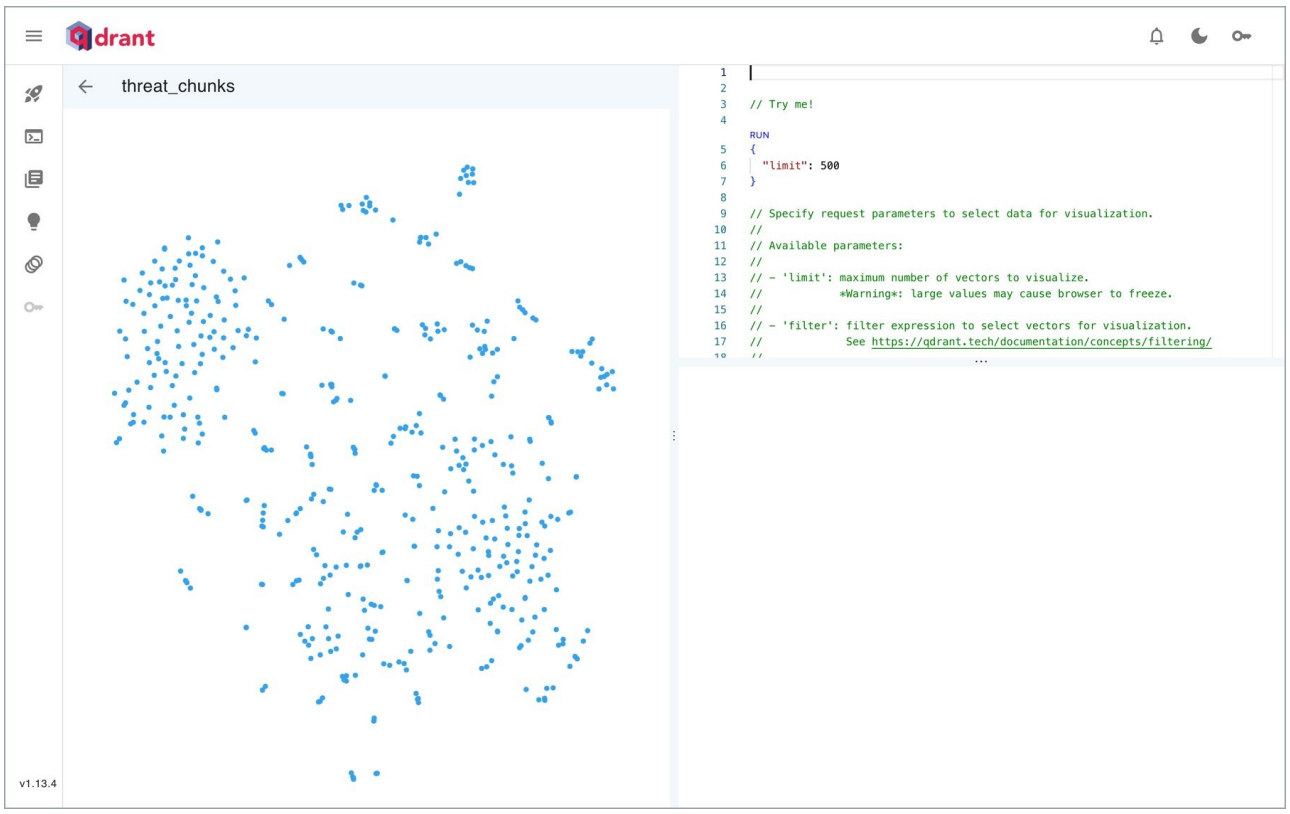


Figure 34. Qdrant's own 2-D projection of 500 stored vectors. Chunks from similar reports form clusters.

Qdrant REST API

```
$ curl -s localhost:6333/   # .version
1.13.4
$ curl -s localhost:6333/collections/threat_chunks   # selected fields of .result
{
  "status": "green",
  "points_count": 796,
  "vectors": {
    "size": 64,
    "distance": "Cosine"
  },
  "hnsw_config": {
    "m": 16,
    "ef_construct": 100,
    "full_scan_threshold": 10000
  },
  "payload_schema": {
    "date_ts": "integer",
    "report_id": "keyword",
    "actors": "keyword",
    "tlp": "keyword"
  }
}
$ curl -s -X POST localhost:6333/collections/threat_chunks/points/count -H 'content-type:
    application/json' \
    -d '{"filter":{"must":[{"key":"source","match":{"value":"abuse.ch Feodo Tracker"}}]},
    "exact":true}'
{"result":{"count":7},"status":"ok","time":0.001856844}
```

6. One Indicator Across the Four Databases

To show that the four databases hold one consistent picture, Table 3 follows a single real indicator through the system. 50.16.16.211 is a QakBot C2 server listed by abuse.ch Feodo Tracker in today's snapshot.

Where	What it holds for 50.16.16.211	Evidence
Redis	Key `ioc:ip:50.16.16.211`: severity critical, QakBot, source abuse.ch Feodo Tracker, TTL 604,800 s (7 days)	Figure 31, Section 5.3
Neo4j	Infrastructure node 50.16.16.211 –[HOSTS]→ QakBot (S0650), which three threat actors USE	Figure 25
MongoDB	In iocs[] and the body of report feed-feodo; two refused requests in blocked_requests, expiring 6 Nov 2026	Figures 29, 30
Qdrant	Chunks of feed-feodo (7 points with source abuse.ch Feodo Tracker), returned as passages C1 and C4	Figure 6, Section 5.4
Enforcement API	`/api/enforce/check` answers blocked, critical; requests from the address get HTTP 403	Section 4.1
GraphRAG	The answer cites G7 (HOSTS edge) and I4 (live, critical, 6 d 23 h left)	Figures 3–5
Integrity check	Every report in MongoDB has a Report node and Qdrant chunks; every Redis IOC traces to the graph or a report	Figure 20, Section 7

Table 3. One indicator traced through the four databases, the API and GraphRAG.

7. Tests and Evaluation on the Live System

All test tiers were run on 30 October 2026 against the same running stack used for the screenshots. The integration tests call the real API, which uses the live stores, and they delete everything they write.

Tier	Command	Result
Unit tests (no database)	`make test`	18 passed in 1.46 s
Integration tests (live API and stores)	`make test-integration`	16 passed in 2.66 s
Acceptance evaluation	`make eval`	6 questions; gold edges 100%, gold reports 100%
Cross-model integrity	part of `make eval`	5 / 5 rules pass

Table 4. Test tiers run on the live system.

```
$ make test   # 2026-10-30 20:57 IST
cd backend && ../.venv/bin/python -m pytest -q
..................                                                       [100%]
18 passed, 16 deselected in 1.46s

$ make test-integration   # 2026-10-30 21:08 IST
cd backend && ../.venv/bin/python -m pytest -q -m integration
................                                                         [100%]
16 passed, 18 deselected in 2.66s
```

7.1 Service-level objectives

Objective	Measured	Target	Status
Redis blocklist GET, p99	1.27 ms	< 5 ms	met
Neo4j traversal, p95 (1 / 2 / 3 hops)	12.69 / 21.4 / 261.79 ms	< 500 ms	met
GraphRAG end to end with the LLM, p95	10.6 s	< 15 s	met
Blocklist freshness	live before expiry, gone after	TTL honoured	met
Cross-model integrity	250 reports = 250 Report nodes = 250 reports in Qdrant	no violations	met

Table 5. Service-level objectives measured by the evaluation on the live system.

These checks cover the system at its current size. The Final Report (Section 9) measures the same objectives as the corpus grows from 1× to 200× and under concurrent load.

7.2 Acceptance questions

Each acceptance question has gold evidence: the graph edges and reports a correct answer must use. The evaluation asks each question through the full GraphRAG pipeline with the local model, and checks that the gold evidence was retrieved and that the answer is grounded.

Question	Gold edges	Gold reports	Citation coverage	Latency
Which threat actors share command-and-control infrastructure, and what malware was hosted there?	2/2	1/1	80%	9.2 s
How is APT29 connected to supply-chain compromises, and which reports describe SUNBURST?	1/1	1/1	100%	9.8 s

Question	Gold edges	Gold reports	Citation coverage	Latency
Who else uses Cobalt Strike besides FIN7, and do they share any domains?	2/2	1/1	50%	6.4 s
What destructive campaigns is Sandworm responsible for, and does it share any infrastructure with APT28?	2/2	1/1	100%	6.4 s
Find activity similar to spearphishing against government and political organizations.	0/0	1/1	80%	10.6 s
Why is Mimikatz a weak attribution signal, and which actors still use it?	2/2	1/1	50%	10.5 s

Table 6. Acceptance questions on 30 October 2026 (llama3.2:3b). Gold edge recall 1.00, gold report recall 1.00, mean citation coverage 77%. The spearphishing question is a similarity question with no gold edges.

8. Submission Contents

Item	File	Contents
This document	`review3/Review3_Submission_GraphRAG_CTI_Platform.pdf` (and .docx)	Working system and live databases
Final report	`review3/Final_Report_GraphRAG_CTI_Platform.pdf`	Design, implementation, testing, performance evaluation
User manual	`review3/User_Manual_GraphRAG_CTI_Platform.pdf`	Installation, every console page, troubleshooting
Presentation	`review3/Presentation_GraphRAG_CTI_Platform.pptx` (and .pdf)	Slides with speaker notes
Demo runbook	`review3/DEMO_RUNBOOK.md`	Live demo script and viva preparation
Source code	`backend/`, `frontend/`, github.com/Shivansharmaaaa/graphrag	API, GraphRAG, ingest, console, tests
Evidence for this document	`review3/submission/screenshots/`, `review3/submission/logs/`	33 original screenshots and the terminal output

Table 7. Review III submission.

The same platform also runs on the database vendors' hosted services: Neo4j AuraDB, MongoDB Atlas, Redis Cloud and Qdrant Cloud. That deployment and its verification on 27 October 2026 are documented in the Final Report (Sections 6.8 and 8.2). This document uses the local deployment.

Conclusion

On 30 October 2026 the platform ran end to end on four NoSQL databases loaded with that day's public threat intelligence. Its answers cited real graph edges, passages and blocklist states. It refused API requests from a live botnet C2 address and logged them with an automatic expiry. It enforced TLP clearance and recorded each answer so it could be replayed against the stores. Every test tier passed, and every service-level objective was met.

Appendix A. Reproducing This Document

The screenshots, the logs and this document can be regenerated on any machine with the project installed (`make install`, Google Chrome, LibreOffice and Pillow).

```
make infra && make dbui && make import-feeds
# terminal 1: the API, with the IP gate demonstrable from this machine
cd backend && TRUST_FORWARDED_FOR=true ../.venv/bin/uvicorn app.main:app --port 8000
# terminal 2: the analyst console
make web
# all screenshots
node review3/capture_submission.js review3/submission/screenshots
# tests and evaluation; their output goes in review3/submission/logs/
make test; make test-integration; make eval
# this document
node review3/generate_submission.js
```